

考虑因果预测与日内调账的微网电力调控策略

摘要

针对含光伏、负荷与储能的微网在日前购电、日内合同调整和变动电价下的协同调度问题，本文建立一套严格遵守信息发布时间的滚动优化框架。首先将附件中以区间终点标记的 10 分钟数据统一映射为自然时段，把功率换算为每槽电量，并以母线平衡、储能效率、功率上限和荷电状态约束构成统一物理层。问题一采用循环日线性规划，以购电费用为主目标、储能吞吐量为次目标，得到典型日购电量 59482.6982 kWh、费用 35126.9489 元，且荷电状态由 6000 kWh 出发并回到同一水平。

对于问题二，本文只使用决策时刻之前的历史数据：以 90 日以内相似日得到负荷和光伏点预测，以 60 日以内的负荷-光伏联合残差构造场景，再用边际经验分位数压缩为保守轨迹并求解线性规划。正式统计覆盖 2025 年 2 月至 12 月共 334 日，1 月仅用于历史启动和储能状态连续推进。固定电价下，无日内调账策略的现金费用为 18764165.93 元，应急购电为 578081.54 kWh。相似日负荷预测的 MAE 和 RMSE 分别为 722.57、904.09 kW；名义 90% 经验区间的实际覆盖率仅为 83.11%，因此本文不把该区间解释为概率保证。

对于问题三，在 0、6、12、18 时接收官方光伏预测，并以最近 60 个已完成日期的同发布时刻误差作因果偏差修正，只重算尚未执行的合同。该策略费用为 16279696.69 元、应急购电为 115458.15 kWh，较问题二节约 2484469.24 元；这一差异同时包含信息改善与更新规则变化，不能全部归因于更新次数。进一步在相同 0 时信息下比较更新集合，基于最新官方预测和已观测前缀构造的 2 小时合成更新相对 6 小时方案节约 708343.30 元，但在另一种合同结算解释下仅节约 127065.26 元，说明建议依赖结算条款和更新成本。

对于问题四，未来电价预测只采用过去 30 日同槽中位数及当日已实现偏差，实际电价仅用于结算。无调账与有调账方案的现金费用分别为 1925.45 万元和 1673.51 万元，后者应急购电为 115938.21 kWh。变价条件下 2 小时方案相对 6 小时方案节约 70.16 万元；若每次额外更新成本高于约 262.59 元，则该增频不再具有净经济优势。通过 54 组基线与单因素实验、逐槽能量平衡、跨日储能连续性和未来信息扰动检验，验证了算法的数值可行性与因果性。本文方法实质上是边际分位轨迹驱动的滚动线性规划，适用于题设数据和结算假设，不等同于完整随机模型预测控制，也不提供样本外概率保证。

关键词：微网调度；因果预测；滚动线性规划；经验分位数；储能；合同调账

1 问题重述与分析

1.1 问题背景与任务拆解

微网由光伏电源、用电负荷、储能装置以及与外部电网相连的购电通道构成。光伏和负荷均具有显著的日内波动，且真实值在决策后才逐步显现；储能能够在时段之间转移电量，却受到容量、功率和往返损耗限制。若日前合同偏低，实时缺口必须以高倍率应急电价补齐；若合同偏高，多余合同电量又可能无法获得完全退款。因此，本题不是单纯的峰谷套利，而是预测误差、合同规则、储能状态和信息更新时间共同作用的序贯决策问题。

已有研究通常将微网不确定性表示为场景、随机过程或滚动预测控制问题 [2, 1]。两层随机模型预测控制可同时处理较慢的计划层与较快的反馈层 [3]，日前市场和储能联合优化也常用随机规划刻画 [5]。滚动时域方法则在新信息到达时重求后续决策 [4]。本题数据量有限，且需要输出指定时段的可核验合同、电池和应急购电结果。为此，本文采用“因果预测-分位轨迹-线性规划-实际执行”的透明结构：不假定未来真实量已知，不把复杂场景树直接塞入控制器，而是用历史联合残差估计风险，再压缩为一条可解释的保守轨迹。

四个问题形成逐层递进关系：

- (1) 问题一给出典型日负荷、光伏和固定电价，要求在日末恢复初始荷电状态。它用于确定物理约束、时间口径和确定性调度基线。
- (2) 问题二将负荷和光伏扩展至全年，但不允许日内调整购电合同。核心是用历史构造次日预测和不确定性余量，并让储能状态跨日连续。
- (3) 问题三在 0、6、12、18 时获得官方光伏预报，可对尚未执行的合同调账。核心是区分“当时已知信息”与“事后真实值”，并按题设规则只结算最终有效合同。
- (4) 问题四把外部电网电价改为随时间变化，要求分别给出无调账和有调账策略。核心是对未来价格也实行因果预测，实际价格只在结算时使用。

从决策目标看，本题至少包含三组相互牵制的量。第一组是合同费用与应急费用：增加日前合同会降低高价应急缺口，却可能产生更多闲置合同；第二组是供电可靠性与光伏消纳：保守净负荷有助于保障负荷，却可能挤占光伏在中午的消纳空间；第三组是当前收益与未来储能价值：过早放电能降低当前槽购电，但会增加后续高价槽的缺口。由此，本文不把单一电量或单一费用作为所有问题的评价标准，而是把现金费用作为优化目标，同时报告应急量、闲置合同、弃光和 SOC 末态，并用终端价值协调短期与后续时段。

问题间比较也需要统一合同。问题二与问题三使用相同固定电价和物理设备，问题四的两个方案使用相同变动电价序列；所有策略从 1 月 1 日同一 SOC 出发，2 月至 12 月采用完全相同的统计日期。只有在这种共同基础上，费用差才具有可比性。另一方面，问题二到问题三不仅增加更新事件，也更换了光伏信息源，因此本文另设“相同 0 时信息”的 0/12/6/2 小时实验来分析更新集合；这一步避免把预报源变化误计为纯频率收益。

1.2 信息集与评价口径

令日期为 d ，每天含 $T = 144$ 个 10 分钟时槽，槽序号为 $t = 0, 1, \dots, 143$ 。在日期 d 的事件时刻 h 做决策时，可用信息集记为 $\mathcal{F}_{d,h}$ 。它包括：所有早于 d 的完整日期数据、当天 h 时之前已经实现的负荷与光伏、截至 h 时已经发布的官方预报，以及变价情形下已经实现的价格。任何 h 时之后的真实负荷、真实光伏和真实价格都不属于 $\mathcal{F}_{d,h}$ 。

为避免首月没有历史而导致模型断裂，计算从 1 月 1 日起推进；历史不足时用附件典型日收缩估计。储能状态从 1 月 1 日连续传递到 12 月 31 日，不在日界处重置。题目要求的结果统计期为 2 月 1 日至 12 月 31 日，共 334 日。因此，文中“年度费用”若无特别说明，均指这 334 日的正式统计，而原始年度负荷、光伏和价格分布图可使用 365 日数据。

评价指标包括现金费用、应急购电量、闲置合同电量、弃光量、储能末态和逐槽物理残差。预测部分报告 MAE、RMSE、MAPE 及经验区间覆盖率，但 MAPE 仅在真实光伏大于 100kW 的样本上计算；若没有有效样本则记为缺失，而不是零误差。时间序列不是相互独立的重复实验，本文不使用未经验证的独立同分布显著性检验。

1.3 总体建模流程

模型首先将输入功率乘以 $\Delta t = 1/6\text{h}$ 得到时槽电量；随后基于 $\mathcal{F}_{d,h}$ 生成负荷、光伏和必要时的价格预测；再由历史联合残差计算经验场景及边际分位轨迹，求解满足储能约束的线性规划；最后用真实量在当前槽执行“余电充电、缺电放电、剩余缺口应急购电”的因果策略。新预报到达时，仅对尚未执行的合同段重复上述过程。典型日输入结构见图1。

这一流程保留了滚动优化的反馈结构，但与完整随机 MPC 不同：控制器输入的是压缩后的确定性分位轨迹，而非带联合概率的完整场景树。这样可以直接追溯每一项输出，同时也意味着经验区间和分位参数只反映本样本中的风险暴露，不能被解释为严格的机会约束保证。

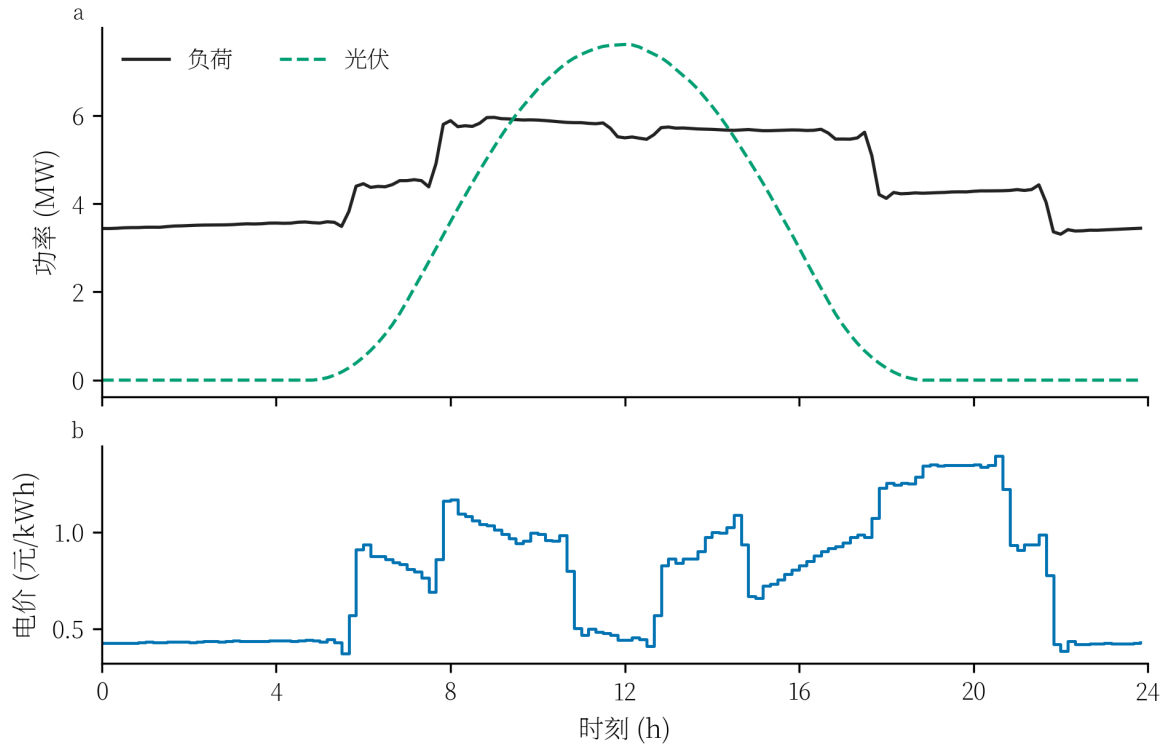


图 1 典型日负荷、光伏与固定电价的日内结构。负荷和光伏峰值并不完全同步，电价阶梯又改变了储能转移能量的价值。

2 模型假设、时间口径与符号

2.1 基本假设

在不改变题面物理含义的前提下作如下假设：

- (1) 微网不能向外部电网反向售电。光伏可被负荷利用、用于充电或弃光；合同电量未使用的部分记为闲置合同电量。
- (2) 储能容量为 12000 kWh，荷电状态限定在 10% 至 90%，即 $1200 \leq S \leq 10800$ kWh；充、放电功率上限均为 5000 kW，单向效率均为 0.9。
- (3) 同一 10 分钟槽不主动同时充电和放电。线性规划若出现数值层面的极小共存量，在执行和审计时按容差处理；真实结果未出现有意义的同时充放。
- (4) 问题一为循环典型日，日末荷电状态等于日初；问题二至四按全年顺序推进，不施加每日循环约束。规划末端价值只用于抑制短视放电，不从报告的现金费用中扣除。
- (5) 相似日、残差、偏差和价格预测均只能使用当前决策时刻以前可得的数据。官方光伏预报按其发布时刻进入信息集，跨年不可得的目标不参与误差计算。

- (6) 题设未给出设备启停、老化、备用容量和配电网潮流限制，因此主目标只计外购合同及应急费用。储能吞吐量作为同费用解之间的次级筛选量，而不是虚构的货币成本。

2.2 10 分钟时间映射

附件中的时刻标签从 00:10 持续到 24:00。本文将标签解释为自然 10 分钟区间的终点：00:10 对应 00:00–00:10，24:00 对应 23:50–24:00。设原始功率为 P_t ，则时槽电量为

$$E_t = P_t \Delta t, \quad \Delta t = \frac{1}{6} \text{ h}. \quad (1)$$

题面给出的 Excel 模板表头保持不变，数值按槽位置写入；论文中的指定时段、SOC 边界点和聚合量统一采用自然区间。这一约定可同时避免把 10 分钟电量误作整小时电量，以及把 24:00 错误解释为下一日首槽。

2.3 统一物理模型

对任意日期和时槽，定义负荷电量 L_t 、可用光伏电量 V_t 、合同购电 g_t 、闲置合同 w_t 、光伏利用 v_t 、充电 x_t 、放电 y_t 、应急购电 u_t 和槽初荷电状态 S_t 。母线平衡为

$$g_t - w_t + v_t + y_t + u_t = L_t + x_t. \quad (2)$$

储能状态递推为

$$S_{t+1} = S_t + \eta_c x_t - \frac{y_t}{\eta_d}, \quad \eta_c = \eta_d = 0.9. \quad (3)$$

约束写为

$$0 \leq v_t \leq V_t, \quad 0 \leq w_t \leq g_t, \quad (4)$$

$$0 \leq x_t \leq P_{\max} \Delta t, \quad 0 \leq y_t \leq P_{\max} \Delta t, \quad (5)$$

$$S_{\min} \leq S_t \leq S_{\max}, \quad u_t \geq 0. \quad (6)$$

其中 $P_{\max} \Delta t = 833.3333 \text{ kWh}$ 。等式(2)把“合同已购买但未使用”的 w_t 明确列出，因而合同电量、实际供能和现金费用不会混淆；式(3)在充放电两侧分别计入效率，往返效率为 0.81。

2.4 主要符号

全文主要符号、单位和决策角色汇总于表1。特别地，功率变量只在输入和图中使用 kW，进入优化器后统一乘以时长成为 kWh；SOC 也是 kWh，因而式(2)和式(3)两侧量纲一致。价格乘以合同或应急电量后得到元。这个转换看似简单，却决定了 5000 kW 功率上限在单槽中应为 833.3333 kWh，而不是 5000 kWh。

表 1 主要符号及单位

符号	单位	含义
d, t, h	–	日期、10 分钟时槽、预报或调账发布时刻
L_t, V_t	kWh	时槽负荷电量、可用光伏电量
g_t, w_t	kWh	合同购电量、闲置合同电量
v_t, x_t, y_t, u_t	kWh	光伏利用、充电、放电、应急购电
S_t	kWh	时槽初荷电状态
c_t	元/kWh	外部电网基准价格或实际价格
p_t, q_t	kWh	0 时不可变基准合同、最终有效合同
α	–	保守净负荷经验分位水平，主方案固定为 0.8
$\mathcal{F}_{d,h}$	–	日期 d 在时刻 h 可获得的信息集

式(2)还区分了合同购电 g_t 与真正流入微网的 $g_t - w_t$ 。如果省略 w_t ，当合同供给超过负荷、充电能力和光伏消纳需求时，模型要么被迫虚构用电，要么错误允许向外送电。显式设置闲置合同变量后，过购不会破坏能量平衡，同时仍按合同规则计费。类似地，弃光量为 $V_t - v_t$ ，它与闲置合同来自不同能源侧，二者不能合并为一个“浪费”变量。

对于全年策略，日末 SOC 不是固定参数，而是下一日的日初状态。设第 d 日末为 $S_{d,144}$ ，则连接条件为

$$S_{d+1,0} = S_{d,144}. \quad (7)$$

这一约束使 1 月的预测和调度虽然不进入正式费用统计，却仍会通过 SOC 影响 2 月 1 日。若每天强行重置到 6000 kWh，相当于在午夜无成本注入或抽走能量，会系统性扭曲年度费用。

3 问题一：典型日循环优化

3.1 确定性线性规划

问题一的负荷、光伏和电价均已知，且不允许应急购电。令购电价格为 c_t ，主目标为

$$\min C_1 = \sum_{t=0}^{143} c_t g_t, \quad (8)$$

并在式(2)–(6)基础上增加 $u_t = 0$ 、 $S_0 = S_{144} = 6000$ 。该模型是线性规划。由于同一最优费用下可能存在多种等价储能轨迹，先求得最小费用 C_1^* ，再在

$$C_1 \leq C_1^* + 10^{-8} \max\{1, |C_1^*|\} \quad (9)$$

的容差内最小化 $\sum_t (x_t + y_t)$ 。因此，报告的费用具有主目标最优性，而储能轨迹是低吞吐代表解，不宣称是唯一最优轨迹。

3.2 调度结果

求解得到总购电量 59482.698167 kWh，现金费用 35126.948941 元；全天充电量和放电量分别为 20740.661758 与 16799.936024 kWh，弃光为 0。初末 SOC 均为 6000 kWh，全天最小与最大 SOC 恰为 1200 和 10800 kWh。逐槽母线平衡最大绝对残差为 9.10×10^{-13} kWh。

图2显示储能在低价时段吸收外购电或光伏，在高价时段放电。由于单向效率为 0.9，只有足够大的价差才能覆盖往返损耗；约束边界处的平段说明容量而非价格判断成为局部瓶颈。日末恢复 6000 kWh 保证该典型日策略不通过透支期末电量来虚降费用。

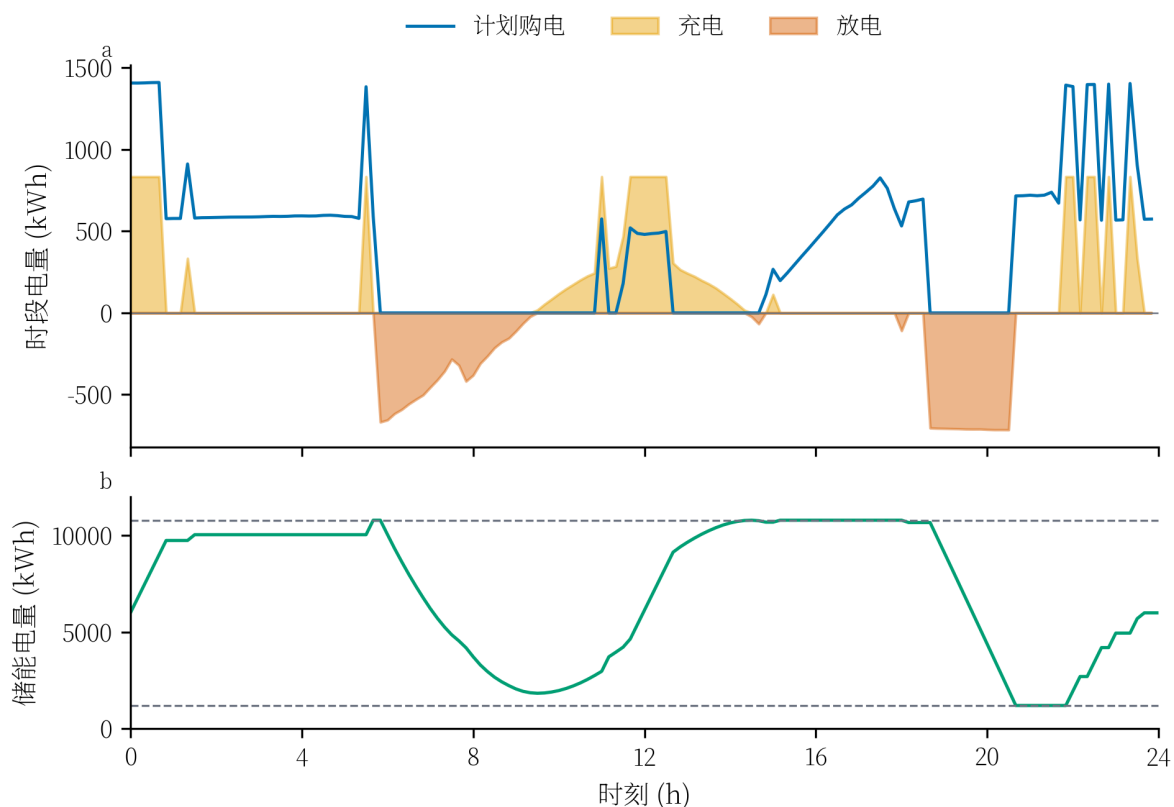


图 2 问题一最优购电、充放电与 SOC 轨迹。SOC 序列含 0 时和 24 时两个边界点。

题目指定的六个 10 分钟购电量列于表2。其中 12:00、16:00 和 18:00 时起始槽需要购电，而 10:00、14:00 和 20:00 时起始槽购电为 0。这里每个数值对应单个 10 分钟槽，不是整小时电量。

表 2 问题一指定时段购电量

自然区间	10:00–10:10	12:00–12:10	14:00–14:10	16:00–16:10	18:00–18:10	20:00–20:10
购电量/kWh	0	480.41245	0	445.43165	531.89405	0

四小时聚合结果见表3。充电量和放电量均由对应 24 个 10 分钟槽求和，SOC 则取自然 0 时和 24 时边界。

表 3 问题一各 4 小时储能电量

时段	充电/kWh	放电/kWh	时段	充电/kWh	放电/kWh
0-4 时	4500.000	0.000	12-16 时	5286.035	91.101
4-8 时	833.333	6365.838	16-20 时	0.000	5780.132
8-12 时	4787.960	1702.997	20-24 时	5333.333	2859.868
0 时 SOC: 6000 kWh			24 时 SOC: 6000 kWh		

若把全部标签整体解释为区间起点并循环平移 10 分钟，购电量几乎不变，而费用变为 35126.848941 元，比主口径低约 0.10 元。这说明时间映射对总量稳健，但费用并非严格不变，故仍应统一自然区间定义。

4 问题二：无日内调账的因果分位轨迹优化

4.1 滚动相似日预测

问题二要求在每天开始前一次性确定全天合同，因而预测只能由过去日期生成。对目标日 d ，取最近 90 日内的历史日期集合 $\mathcal{H}_d$ 。对负荷，目标日与历史日星期属性相同时附加权重 2，否则权重 1；光伏不使用星期加权。时间衰减统一为

$$\omega_{dk} = a_{dk} \exp\left(-\frac{d-k}{60}\right), \quad k \in \mathcal{H}_d, \quad (10)$$

其中 $a_{dk} = 2$ 或 1。历史样本较少时，将加权历史均值向附件典型日收缩。以负荷为例，

$$\hat{L}_{d,t} = \gamma_d \frac{\sum_{k \in \mathcal{H}_d} \omega_{dk} L_{k,t}}{\sum_{k \in \mathcal{H}_d} \omega_{dk}} + (1 - \gamma_d) L_t^{\text{typ}}, \quad \gamma_d = \min\left(1, \frac{|\mathcal{H}_d|}{21}\right). \quad (11)$$

首日没有历史时直接采用典型日；光伏采用相同结构但 $a_{dk} = 1$ 。该收缩使冷启动平滑过渡到滚动历史估计，不需要把未来全年样本先用于训练。

图3展示 365 日净负荷 $P_{\text{load}} - P_{\text{pv}}$ 的季节和日内结构。正负区间交替说明光伏富余与负荷缺口都具有显著时变性；仅用单个年度均值无法形成可执行合同。

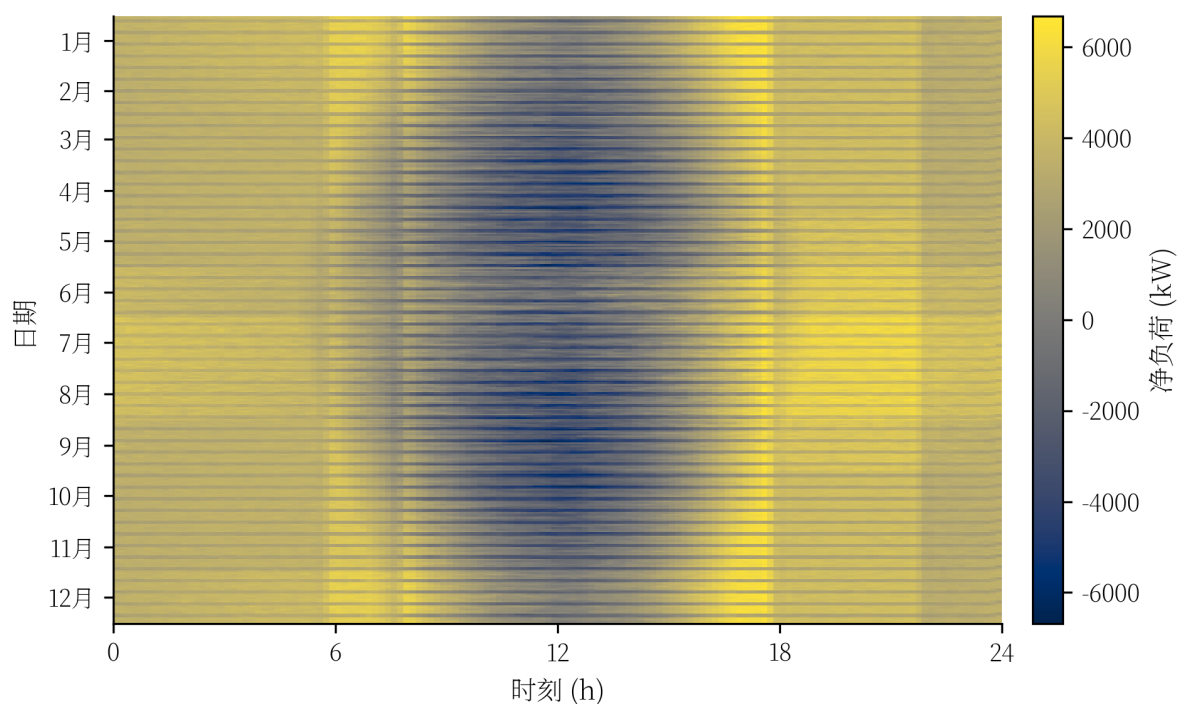


图 3 2025 年 365 日净负荷的季节-日内分布。该图使用全年原始数据，策略费用统计期另为 2 月至 12 月。

在正式统计期上，相似日负荷预测 MAE 为 722.5708kW，RMSE 为 904.0899kW；光伏预测 MAE 为 335.9217kW。图4表明多数点接近 45 度线，但仍存在明显尾部误差。因此，单纯把点预测当作确定真值会低估应急购电风险。

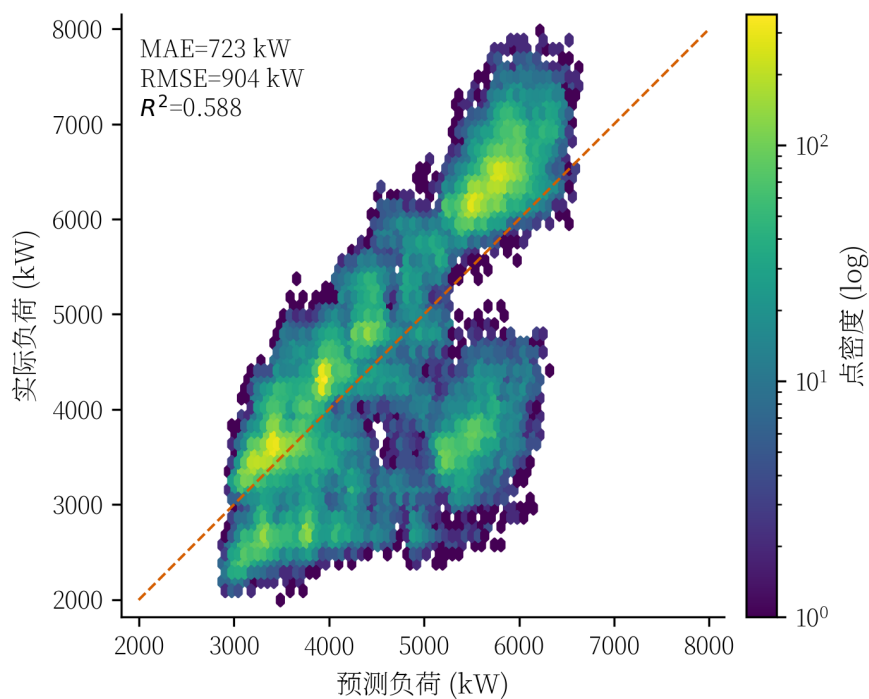


图 4 2 月至 12 月因果相似日负荷预测与真实负荷的比较。颜色表示样本密度，离 45 度线较远的点对应尾部误差。

4.2 联合残差场景与边际分位压缩

定义负荷和光伏残差

$$e_{d,t}^L = L_{d,t} - \hat{L}_{d,t}, \quad e_{d,t}^V = V_{d,t} - \hat{V}_{d,t}. \quad (12)$$

在日期 d 只取最近 60 个已完成日期的残差，将一天的两类残差拼接为 288 维向量，并按各维历史标准差标准化。标准差小于 10^{-9} 时置为 1，避免近常数维放大数值误差。随后用 K -means 将残差日聚为至多 9 类，随机种子固定为 2026，重复初始化 10 次；实际簇数不超过历史样本中唯一残差向量数。空历史退化为一个零残差场景，单条历史保留该样本本身。

设第 s 个场景的负荷和光伏轨迹为 $L_t^{(s)}, V_t^{(s)}$ ，簇权重为 π_s 。对净负荷 $N_t^{(s)} = L_t^{(s)} - V_t^{(s)}$ 取加权经验 α 分位数，对光伏取 $1 - \alpha$ 分位数：

$$\tilde{N}_t = Q_\alpha(\{N_t^{(s)}, \pi_s\}), \quad (13)$$

$$\tilde{V}_t = \max\left\{Q_{1-\alpha}(\{V_t^{(s)}, \pi_s\}), -\tilde{N}_t, 0\right\}, \quad (14)$$

$$\tilde{L}_t = \tilde{V}_t + \tilde{N}_t. \quad (15)$$

经验分位数采用累计权重首次达到阈值的样本，不作线性插值。主方案在计算前固定 $\alpha = 0.8$ 。式(15)分别保守化净负荷和光伏，并通过重构保证 $\tilde{L}_t, \tilde{V}_t \geq 0$ 及二者净差一致。

这种压缩保留了“净需求偏高、可用光伏偏低”的风险方向，却不再保留完整的跨时段联合分布。因此，后续优化应称为边际分位轨迹线性规划，而不是完整随机 MPC。滚动残差给出的名义 90% 经验区间对负荷、光伏的实际覆盖率分别为 83.1109% 和 83.4581%，均低于 90%；它只能作为历史回测描述，不能构成概率保证。

从实现顺序看，每个日期的预测与规划包含五个确定步骤。首先读取不超过当前日期的历史索引，并生成相似日点预测；其次在历史残差上进行退化检查和标准化；再次聚类得到代表残差与经验权重；随后按式(15)逐槽取分位数并重构安全负荷；最后以当天实际 SOC 为初态求解 144 槽 LP。当天结束后才把真实残差加入下一日历史。该顺序保证“先预测、后观察、再更新历史”，不会因为一次性载入全年附件就把未来行用于当前决策。

选择边际分位压缩而没有直接对所有场景逐一设置供电约束，主要基于两点。其一，题目要求连续运行 334 个正式日期并完成 54 组实验，压缩后每次规划规模固定，便于稳定求解和逐槽复核；其二，分位参数 α 直接对应净负荷上侧与光伏下侧的保守方向，能够通过敏感性曲线解释风险偏好。代价是场景之间的时序相关只用于形成代表样本，进入 LP 后被一条轨迹替代。因此，本文把场景聚类视为风险特征提取，而非声称求解了多阶段随机规划。

4.3 日前合同优化与真实执行

在日期 d 的 0 时，用安全轨迹 $(\tilde{L}_t, \tilde{V}_t)$ 代替未知真实轨迹，求解统一物理模型。固定电价下，规划目标写为

$$\min \sum_{t=0}^{143} c_t g_t - \lambda_d S_{144}, \quad \lambda_d = \eta_d \text{median}\{c_t : t \text{ 属于剩余规划域}\}. \quad (16)$$

末端价值 $\lambda_d S_{144}$ 防止算法在一天末尾无代价地放空电池，但报告现金费用时不扣除这项信用。优化完成后合同 g_t 全天不再改变。

真实槽到达时，执行层只使用当前真实负荷和光伏。若光伏及合同供给有余量，则先在功率和 SOC 允许范围内充电，之后的合同剩余计为闲置合同，光伏剩余计为弃光；若有缺口，则先放电，仍不能满足的部分以 5 倍当前电价应急购电。这个优先级是透明的因果反馈规则，而不是用全天真实量重新优化。

规划层与执行层分离有一个重要含义：LP 给出的充放电计划服务于安全轨迹，真实执行动作则必须服从已实现的供需和当前 SOC。若真实光伏高于保守轨迹，额外余电优先充电；若真实负荷高于轨迹，电池可能比计划更早放电。因而“计划合同总量”与“实际购电供能量”不完全相同，现金费用也不能由计划目标值直接替代，必须逐槽按真实执行和应急量重新结算。

4.4 年度结果与基线比较

从 2 月 1 日至 12 月 31 日，无日内调账主策略现金费用为 18764165.9334 元，其中普通合同结算为 15258616.4937 元，应急费用为 3505549.4397 元；应急购电量为 578081.5436 kWh，闲置合同电量为 4123779.9049 kWh，弃光为 1950263.9848 kWh。年末 SOC 为 10609.7713 kWh。每日费用和应急量具有明显异质性，见图5，图中的 7 日中心均线仅用于事后描述，不进入任何日前预测。

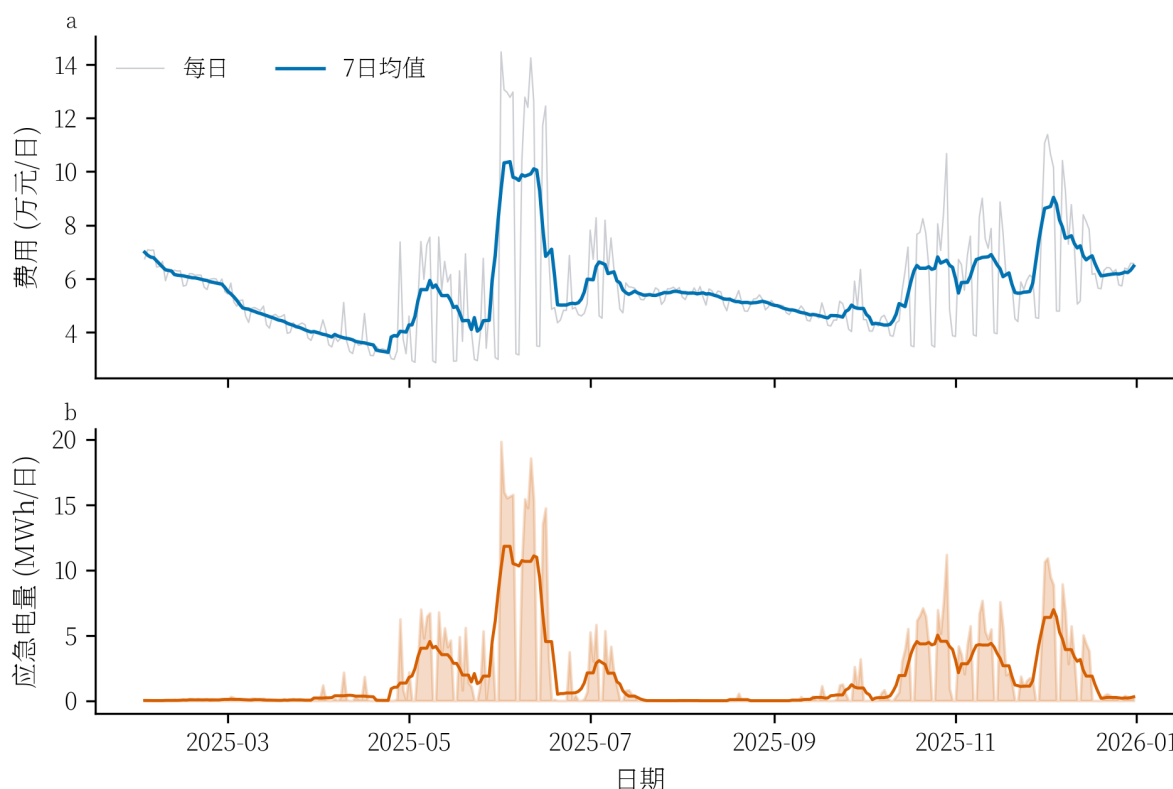


图 5 问题二 334 日现金费用与应急购电量。原始日值显示尾部风险，中心均线不作为模型输入。

为判断保守轨迹的作用，比较点预测、典型日和无储能三类基线。表4显示，点预测

和典型日方案都因应急购电显著增加而费用较高。无储能方案费用低于这两项，却仍高于保守轨迹主策略；它的应急量与主策略相近，但弃光更高，说明储能和合同保守度必须联合考察，不能只看应急电量。

表 4 问题二主策略与基线的 334 日结果

策略	现金费用/元	应急购电/kWh	弃光/kWh
保守分位轨迹	18,764,165.93	578,081.54	1,950,263.98
点预测	27,574,292.97	2,685,630.97	1,618,696.22
典型日	27,753,076.30	2,724,419.83	1,773,185.71
无储能	21,799,176.46	609,913.86	3,058,159.92

图6进一步把合同结算与应急费用分开。主策略普通合同部分较高，是对预测尾部风险的事前支付；点预测和典型日虽然减少合同购买，却把更多缺口暴露在 5 倍应急价格下。无储能方案没有充放损耗，但中午光伏难以跨时段转移，其弃光约 305.82 万 kWh，显著高于有储能主策略。由此，主策略的优势不是单纯“买得更多”或“电池越大越好”，而是合同保守度与储能转移共同改变了高价缺口的位置和规模。

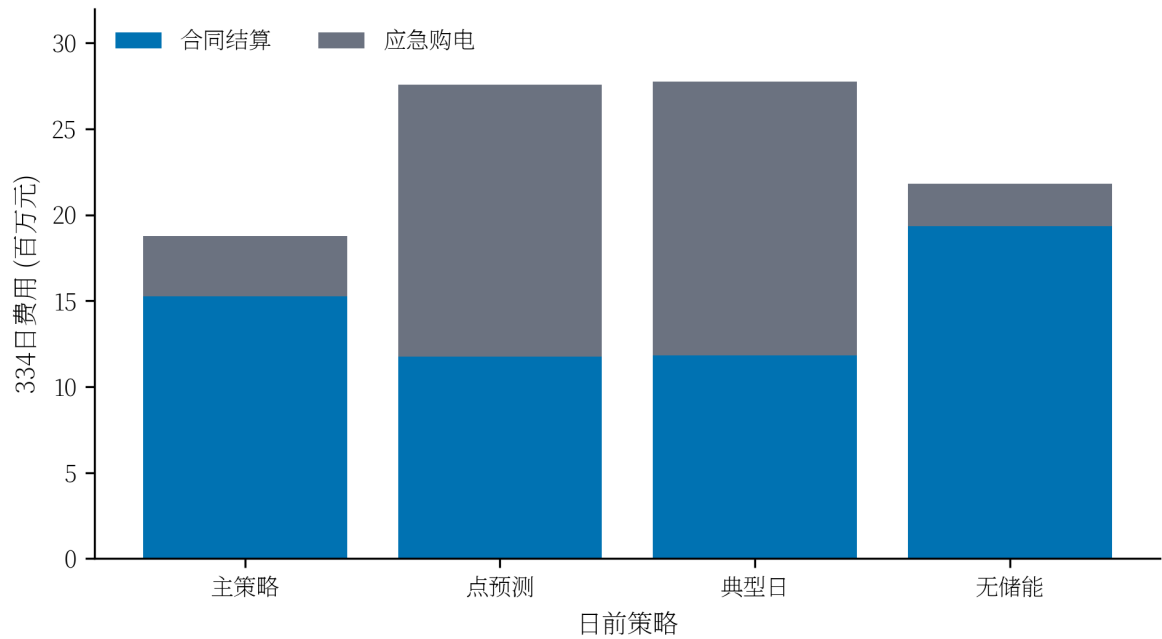


图 6 问题二不同基线的合同费用与应急费用分解。柱高为 334 日总额，不表示抽样均值的置信区间。

四个代表日的指定购电、储能和应急表分别列于附录A。代表日结果同时显示一个重要边界：策略在总体上降低风险，并不意味着每个日期都严格优于其他策略；这种日期异质性将在问题三中继续讨论。

5 问题三：日内光伏预报与合同调整

5.1 官方预报的因果处理

附件给出 0、6、12、18 时发布的逐小时光伏预报。对发布时刻 $h \in \{0, 6, 12, 18\}$ ，先将未来整点预报裁剪为非负，再以当前时刻已经观测的光伏端点为锚，线性插值到 10 分钟网格。1 月 1 日 0 时没有前一已观测端点，锚值取 0。这样既避免光伏出现负值，也防止插值曲线在更新时刻与真实已观测前缀脱节。

令 $F_{d,h,\ell}$ 为日 d 在时刻 h 对提前 ℓ 小时目标的官方预报，真实值为 $V_{d,h+\ell}$ 。偏差样本定义为

$$r_{d,h,\ell} = V_{d,h+\ell} - F_{d,h,\ell}. \quad (17)$$

在日期 d 进行修正时，只取最近 60 个已完成日期的同一发布时刻残差均值，不能使用当天尚未完成的残差。经非负裁剪的修正预报为

$$\hat{V}_{d,h,\ell}^{\text{off}} = \max \left\{ 0, F_{d,h,\ell} + \frac{1}{|\mathcal{R}_{d,h}|} \sum_{k \in \mathcal{R}_{d,h}} r_{k,h,\ell} \right\}. \quad (18)$$

若历史为空，则修正量为 0。

图7给出原始预报误差随发布时间和提前期的变化。对相同的当日剩余目标，0、6、12、18 时原始 MAE 分别为 186.7347、129.0903、35.1985 和 0.00036 kW，修正后分别为 188.1883、130.9920、35.5854 和 0.00051 kW。修正降低了平均偏差的绝对值，但 MAE 并未改善；18 时剩余目标主要处于夜间，满足真实光伏大于 100 kW 的 MAPE 有效样本数为 0，故不报告 MAPE 数值。该结果说明“消除均值偏差”不等同于“降低绝对误差”。

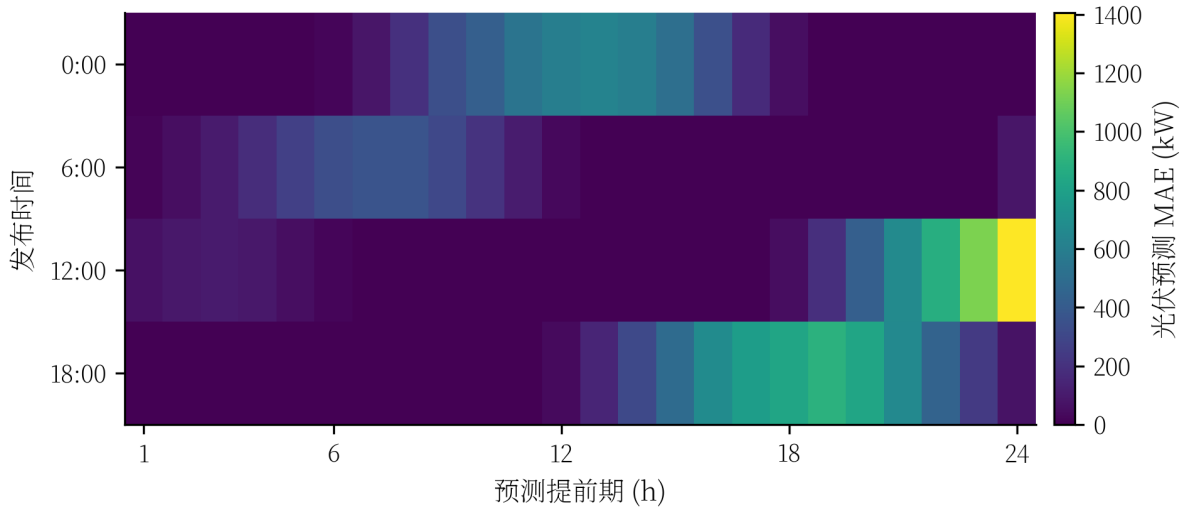


图 7 官方光伏预报 MAE 随发布时刻与提前期的变化。跨年不可得目标被排除，颜色不代表概率置信度。

5.2 滚动合同与结算模型

0 时先生成全天合同 p_t 。每次更新到达后，对 $t \geq 6h$ 、 $12h$ 或 $18h$ 的尚未执行时槽重新求解，已执行合同和实际能量流保持不变。规划中的负荷预测仍来自相似日模型；安全光伏轨迹用更新后的官方预报替代点预测，并继续使用过去残差形成保守余量。当天 SOC 从前一事件末端传递到下一事件，不因重规划而重置。

设 q_t 为某时槽最后一次更新后真正执行的合同。主结算解释以 0 时合同 p_t 为不可变基准，并按最终合同只结算一次：

$$C_t = c_t [\min(p_t, q_t) + 1.5 (q_t - p_t)^+ + 0.5 (p_t - q_t)^+] + 5c_t u_t. \quad (19)$$

其中下调合同对应半价退还的净效果；多次中间修改不逐次重复收费。为检验题意解释的影响，再对同一物理执行轨迹计算

$$C_t^{\text{alt}} = c_t [p_t + 1.5 (q_t - p_t)^+ + 0.5 (p_t - q_t)^+] + 5c_t u_t. \quad (20)$$

式(20)把下调量作为额外费用而非退款。它只用于同策略重新计费，没有在该规则下重新优化；在允许合同闲置且无退款收益的情况下，下调动作甚至可能被不下调严格支配。因此，两种费用必须分别解释，不能混称为同一个最优问题。

以 $p_t = 10 \text{ kWh}$ 、 $c_t = 1 \text{ 元/kWh}$ 且无应急购电为例，最终合同 $q_t = 5, 10, 15 \text{ kWh}$ 时，主结算费用分别为 7.5、10 和 17.5 元；替代结算分别为 12.5、10 和 17.5 元。该例说明两式只在下调区间不同：主式把未执行的 5 kWh 基准合同按半价净退款处理，替代式则在原合同上再加半价下调费用。若不先明确这一差别，同一合同轨迹会得到不同的频率建议。

滚动求解还必须保持事件间状态一致。设相邻更新时刻为 $h_j < h_{j+1}$ ，在 h_j 只求解尚未执行的剩余段；区间 $[h_j, h_{j+1})$ 按真实值执行后，得到的 SOC 是下一次 LP 的固定初态。已执行合同、充放电和应急量均不回写。因而最终日费用可以按 144 个槽各自的最后有效合同相加，而不能把四次规划目标相加。

图8展示 6 月 21 日合同随预报更新发生有方向的改变。图中的实际净负荷尚未扣除储能动作，因此不能把合同曲线与净负荷曲线的差直接解释为应急量或弃光量。

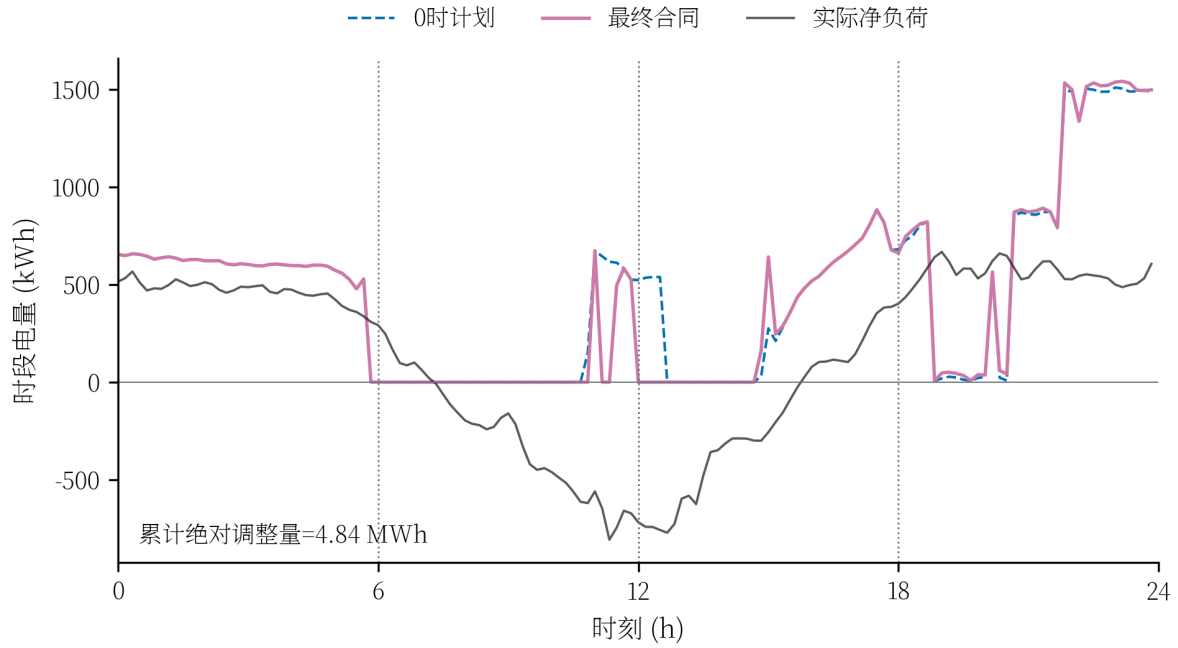


图 8 6 月 21 日 0 时初始合同与日内最终合同。竖线为 6、12、18 时更新事件。

5.3 主策略结果

问题三 334 日现金费用为 16279696.6905 元，应急购电量为 115458.1457 kWh，闲置合同为 3093394.8432 kWh，弃光为 1841583.5122 kWh，年末 SOC 为 10649.4342 kWh。相对问题二，现金费用减少 2484469.2430 元，应急购电减少约 80.03%。但这项差异同时改变了光伏信息来源和合同更新规则，并非“只增加更新次数”的纯频率效应。

四个代表日中，3 月 20 日、6 月 21 日和 9 月 23 日费用下降，而 12 月 21 日费用由 63120.3057 元上升为 65731.5948 元，应急购电也从 382.2268 增至 979.7444 kWh。因此，年度改善不能被写成逐日占优。详细代表日购电、储能与应急结果见附录 A。

5.4 相同 0 时信息下的更新频率分析

为尽量分离“0 时信息不同”的影响，固定 0 时采用问题三的统一官方信息，比较只在 0 时、0/12 时、0/6/12/18 时和每 2 小时更新四组方案。2 小时情形并非附件提供的新官方预报：在非官方发布时刻，模型沿用最近一次已发布预报，并以当天已经实现的最多 18 个 10 分钟槽偏差构造 3 小时指数衰减的合成短临更新；该更新只调整安全光伏，同时保持安全负荷轨迹，不重新标定场景概率。

四组费用及两种结算的对照列于表 5。负的“相对 6 小时节约”表示候选方案更贵，而不是负费用。

表 5 固定电价下相同 0 时信息的更新集合比较

更新集合	主结算费用/元	相对 6 小时节约/元	替代结算相对 6 小时节约/元
仅 0 时	17,613,329.64	-1,333,632.95	-537,280.34
0、12 时	17,143,902.65	-864,205.96	-659,112.90
每 6 小时	16,279,696.69	0	0
每 2 小时合成更新	15,571,353.39	708,343.30	127,065.26

2 小时方案在主结算下相对 6 小时方案节约 4.3511%，全年增加 2672 次更新。若每次额外更新的综合通信、计算和运维成本记为 k ，则其净收益为

$$B_{2h} = 708343.3002 - 2672k. \quad (21)$$

故盈亏平衡阈值为 $k^* = 265.0985$ 元/次。四个代表日在主结算口径下均不劣，但替代结算下总节约仅约 12.71 万元，且仅 0 时与 12 小时方案的排序会反转。图9直观显示结算解释对更新价值的影响，因此合理建议是“在主结算成立且每次增量成本低于阈值时采用 2 小时合成更新”，而不是无条件追求最高更新频率。

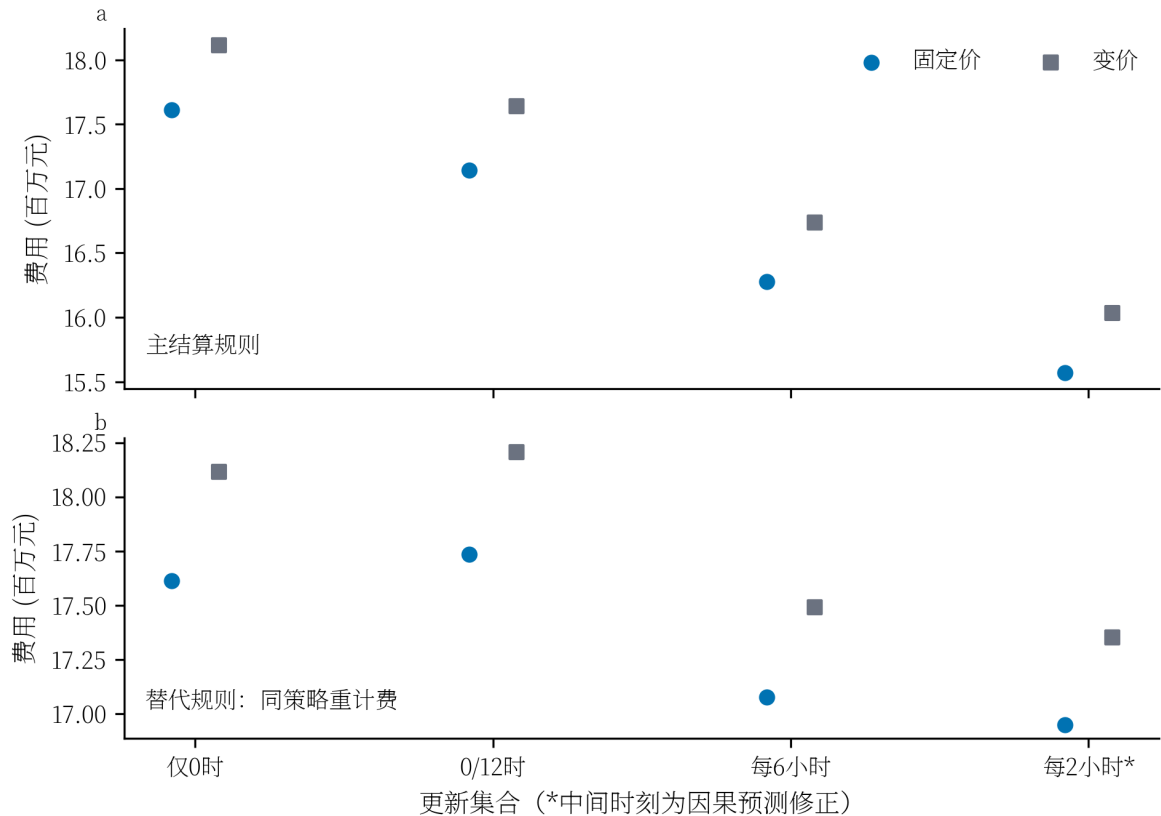


图 9 固定与变动电价下不同更新集合的总费用，分别按主结算和替代结算重计。2 小时方案使用因果合成更新。

6 问题四：变动电价下的滚动调度

6.1 未来价格的因果预测

问题四的实际电价随日期和时槽变化。若在规划时直接读取当天未来真实价格，会产生明显的未来信息泄漏。本文对日期 d 、槽 t 的 0 时价格预测取过去 30 日同槽价格的中位数：

$$\hat{c}_{d,0,t} = \text{median} \{c_{k,t} : \max(0, d-30) \leq k < d\}. \quad (22)$$

首日没有历史时使用附件典型价格。日内时刻 h 重新规划时，计算当天已经实现的最近至多 18 槽价格偏差中位数 $b_{d,h}$ ，并按 3 小时尺度向未来衰减：

$$\hat{c}_{d,h,t} = \max \left\{ 10^{-4}, \hat{c}_{d,0,t} + b_{d,h} \exp \left[-\frac{t-t_h}{18} \right] \right\}, \quad t \geq t_h. \quad (23)$$

其中 $t_h = 6h$ 。规划目标使用 $\hat{c}$ ，真实 c 仅用于合同和应急费用结算。图10显示各月价格分布具有离散性和尾部，故单一月均价不足以描述调度机会；箱线图外的离群点均被保留。

中位数而非均值用于 0 时价格预测，是因为月内价格分布包含尾部，少数尖峰会明显拉动均值。该选择并不意味着忽略尖峰：安全供能仍由负荷和光伏分位轨迹控制，而价格中位数只决定有限电池能量在剩余时槽间的相对配置。当日偏差项使整体价格水平随已实现数据上移或下移，指数衰减则避免把刚刚发生的局部异常永久外推到全天。

在实际执行时，储能动作与真实价格的相关性只能作为事后描述，不能反向证明控制器预知价格。规划时可利用的是 $\hat{c}_{d,h,t}$ ；真实 $c_{d,t}$ 在槽结束时进入结算。若实际高价尖峰未被历史中位数和已实现偏差预见，策略可能保留不足的 SOC，这正是个别代表日有调账方案费用反而上升的可能机制之一。

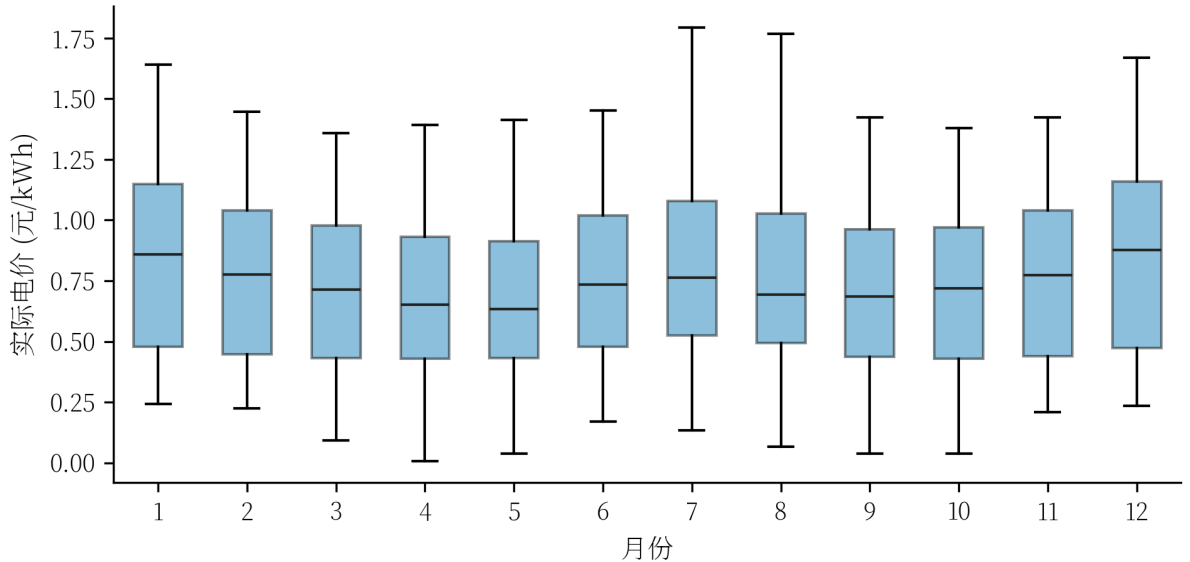


图 10 2025 年各月 10 分钟电价分布。箱体为四分位区间，须线为 1.5 倍四分位距，离群点未删除。

6.2 两种策略及结果

问题四的无调账方案沿用问题二的相似日与分位轨迹，只把固定价目标替换为式(22)的预测价格；有调账方案沿用问题三的 0、6、12、18 时更新机制，并在每次滚动求解时使用式(23)。两者执行层均只读取当前真实量，储能跨日连续。

表6给出正式统计结果。有调账方案比无调账方案节约 2519380.1148 元，应急购电减少 459960.1389 kWh。若把问题二和问题三也纳入同一图，固定价与变价都呈现“有调账总体费用更低”的结果，见图11。但是，固定价的 $q_2 \rightarrow q_3$ 和变价的 $q_{4_2} \rightarrow q_{4_3}$ 都同时改变了信息与更新规则，不能当作单因素频率效应。

表 6 问题四两个方案的 334 日结果

方案	现金费用/元	应急/kWh	闲置合同/kWh	弃光/kWh
无调账	19,254,498.44	575,898.35	4,100,533.58	1,969,275.57
有调账	16,735,118.32	115,938.21	3,085,161.95	1,848,622.33

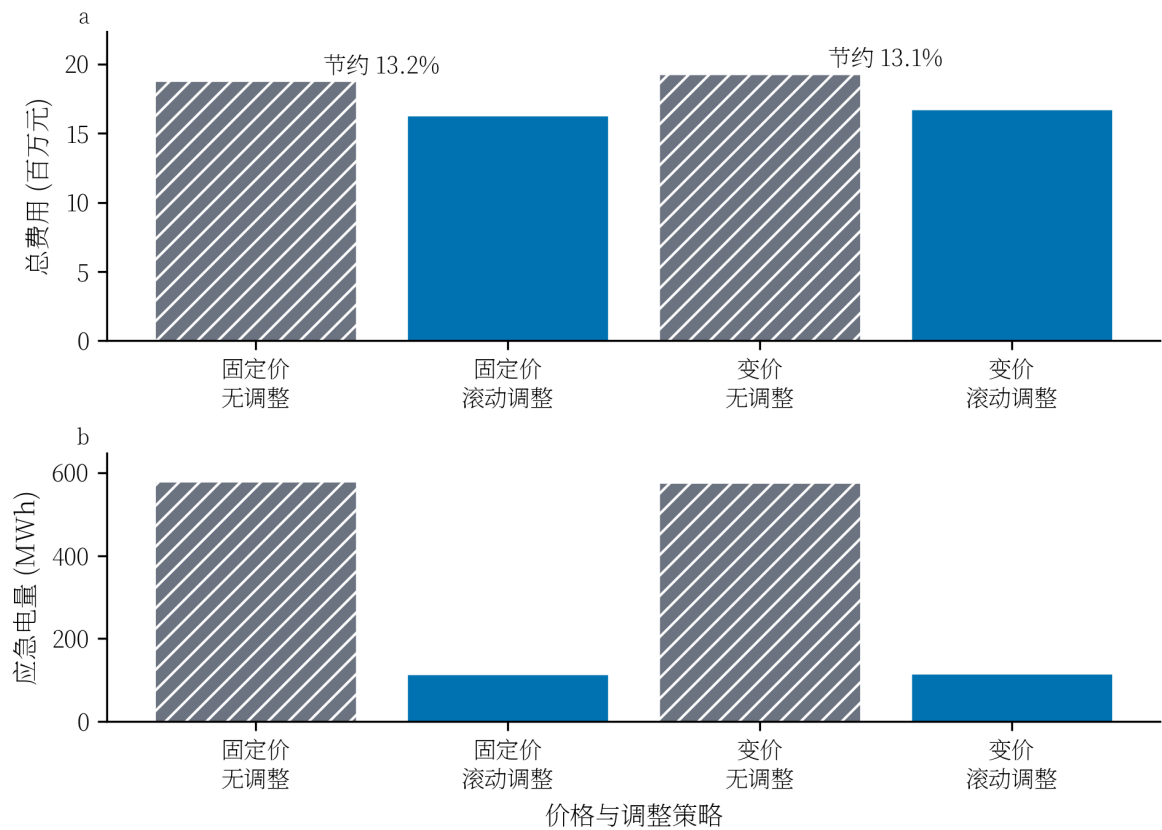


图 11 固定价与变动电价下四个主策略的 334 日费用和应急购电量。

代表日同样并非全部改善：12 月 21 日无调账方案费用为 74025.4512 元，有调账方案反而为 77842.3118 元。其应急电量由 382.2268 变为 978.6827 kWh。这说明滚动预测在总体上更有效，却可能在个别日期因预报误差、储能边界和价格预测偏差叠加而失利。两方案的完整代表日指定表见附录A和配套工作簿。

6.3 变价条件下的更新频率建议

在相同 0 时信息下，变价 2 小时合成更新方案费用为 16033486.6253 元，6 小时方案为 16735118.3246 元，前者节约 701631.6992 元，即 4.1926%。两方案相差 2672 次更新，故净收益为

$$B_{2h}^{\text{var}} = 701631.6992 - 2672k, \quad (24)$$

盈亏平衡阈值为 262.5867 元/次。若采用替代结算式(20)对同一策略重计费，节约额缩小为 139845.6748 元，不能把主结算阈值直接移用于该口径。

因此，问题四的建议为：在题设主结算解释成立、能够可靠取得最新官方预报和已观测前缀，且单次增量更新成本低于约 262.59 元时，可采用 2 小时合成更新；否则 6 小时官方发布节奏具有更稳健的实施性。2 小时结果仅表示本题数据上的因果短临更新价值，不代表系统获得了额外官方预报产品。

7 模型检验、敏感性与稳健性分析

7.1 物理可行性与因果性检验

对问题一及四个年度主策略逐槽检查式(2)、式(3)、SOC 边界、充放电功率、非负性、同时充放和跨日连续性。问题一最大母线平衡残差为 9.10×10^{-13} kWh；四个主策略正式统计期的最大平衡残差不超过 3.70×10^{-13} kWh，独立逐槽重算的最大 SOC 递推残差约为 4.55×10^{-12} kWh。这些量接近浮点运算精度，表明结果满足模型物理方程。

因果性检验在第 0、1 和 364 日的 0、2、...、22 时，对未来负荷、光伏、价格和未发布预报分别作 36 组扰动。当前预测、价格估计和当前 LP 计划的最大差均为 0；已执行前缀和 SOC 边界也不受未来扰动影响。这验证了实现遵守信息集 $\mathcal{F}_{d,h}$ 。该检验说明“没有使用被扰动的未来数据”，但不是对所有可能代码路径的形式化证明。

7.2 54 组实验设计

实验共 54 组，其中 16 组为固定价/变价的安全轨迹、点预测、典型日、无储能，以及 0/12/6/2 小时更新基线；另有 38 组单因素实验。单因素范围包括：

- 保守分位参数 $\alpha = 0.70, 0.75, 0.80, 0.85, 0.90, 0.95$ ；
- 残差聚类上限 5、9、12、16，残差历史 30、60、90 日；
- 储能容量和功率分别为主值的 0.8、1.0、1.2 倍；
- 往返效率从 0.81 提高到 0.90，即单向效率改为 $\sqrt{0.9}$ ；
- 应急电价倍率 3、4、5、6、8。

容量改变时，SOC 下限、上限和初态仍分别为容量的 10%、90% 和 50%，而功率与效率保持不变；功率实验则不改变容量。应急倍率实验固定物理策略和 $\alpha = 0.8$ ，只重新计算费用暴露，因此不能称为各倍率下重新寻优的最优策略。

7.3 保守参数与设备参数

图12展示 α 对费用和应急量的权衡。固定价问题二中， $\alpha = 0.95$ 的样本内费用低于主设定 0.8；变价有调账方案中，0.8 是测试点中的最低费用。由于主参数在查看本年度结果前固定，本文不据此把 0.95 或 0.8 宣称为普适最优参数。样本内曲线的主要价值在于说明保守度改变会同时影响合同过购、应急缺口和弃光，而不是提供无偏超参数选择。

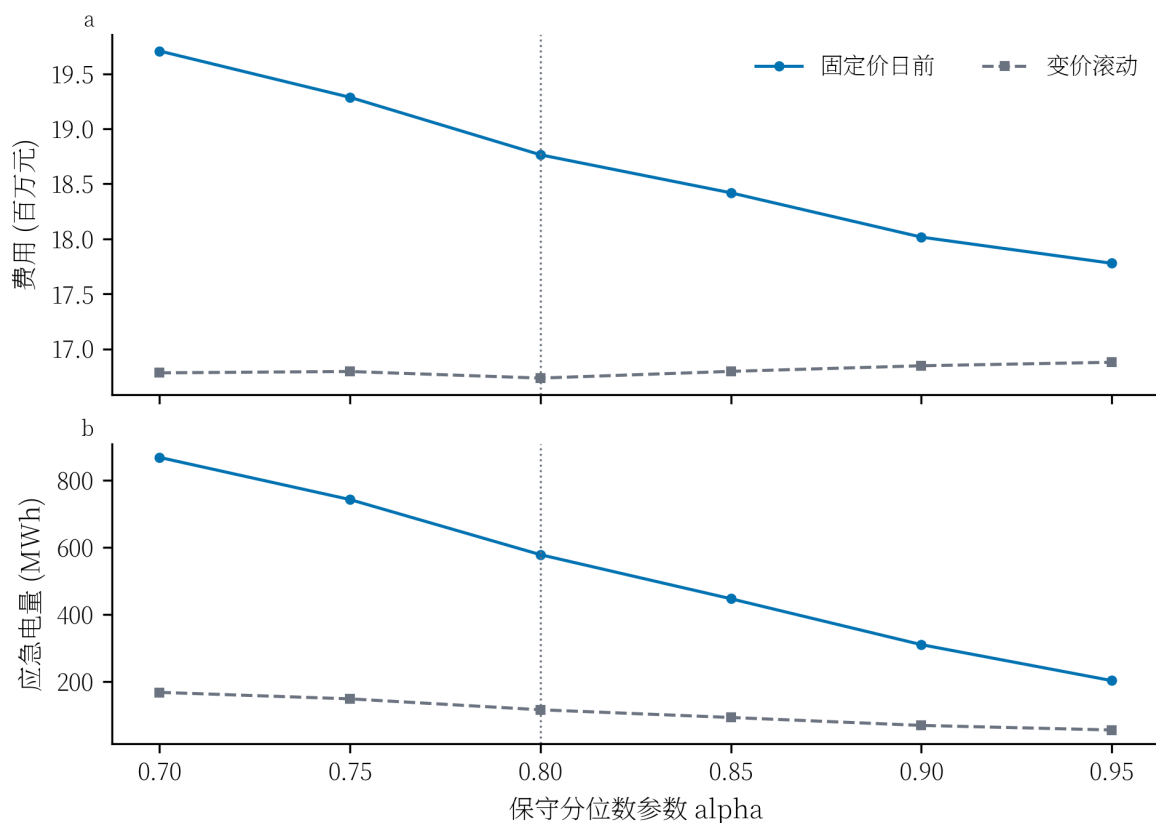


图 12 问题二与问题四有调账策略对保守分位参数的单因素敏感性。主方案 $\alpha = 0.8$ 为事前固定值。

历史窗口 30 日时，问题二和问题四有调账方案费用分别约为 1698.74 万元和 1629.52 万元；90 日窗口更高。这反映 2025 年样本中的近期结构更有用，但不证明短历史在其他年份总是更好。容量增加 20% 后，两方案费用分别约为 1821.87 万元和 1600.59 万元；容量减少 20% 并不必然按比例增加费用，因为合同风险、弃光和终端 SOC 共同变化。

表7列出若干有代表性的单因素结果。历史窗口 30 日对两个方案都较有利，说明 2025 年数据存在较强的近期性；提高往返效率则稳定降低费用。容量增加对问题四有调账方案的收益大于对问题二，原因是变价环境下额外可用能量同时具有峰谷转移和风险缓冲价值。所有比较都只改变表中一个因素，其余参数保持主设定。

表 7 部分单因素实验的 334 日现金费用（万元）

实验设置	问题二无调账	问题四有调账
主设定	1876.42	1673.51
残差历史 30 日	1698.74	1629.52
储能容量增加 20%	1821.87	1600.59
储能功率降至 80%	—	1669.53
往返效率提高到 90%	1832.77	1620.57

问题四有调账方案将功率降至主值的 80% 时，样本内费用约为 1669.53 万元，略低于主方案 1673.51 万元。这一非单调性并不违反物理规律：控制器采用保守轨迹规划和真实值下的贪心执行，并非在完全信息下对全年重新全局优化；更大的设备可行域不会自动保证启发式滚动策略产生更低的实际结算费用。往返效率提高到 90% 后，费用下降到约 1620.57 万元，符合损耗降低的方向性预期。

实验还揭示了初态与年末状态的关系。所有容量实验都从容量的 50% 起步，SOC 上下界同步保持 10% 和 90%；这保证比较不是由绝对初态不一致造成。然而，正式费用从 2 月开始统计，1 月调度已经把不同容量系统推进到各自状态，因此不能简单断言初始 SOC 影响在一个月后完全消失。本文只报告统一启动规则下的可比结果，不对无限期稳态作额外假设。

7.4 结算与时间口径敏感性

同一策略在主结算和替代结算下费用差异明显。问题三主策略由 1627.97 万元增至 1707.60 万元，问题四有调账策略由 1673.51 万元增至 1749.07 万元。更重要的是，2 小时相对 6 小时的收益在替代口径下明显收缩。因此，更新频率结论必须与式(19)绑定，不能只给一个脱离合同条款的百分比。

固定价方案还出现排序反转：主结算下 12 小时方案费用低于仅 0 时方案，而替代重计费下仅 0 时方案约 1761.33 万元，反而低于 12 小时方案约 1773.52 万元。其原因不是物理供能突然变化，而是替代规则对合同下调额外收费。该现象说明，调账价值必须同时报告物理效果和条款效果；仅凭“应急减少”不足以得出经济最优结论。

时间口径实验把典型日全部槽循环平移 10 分钟。购电总量仅有约 2.38×10^{-9} kWh 的浮点差，而费用下降约 0.10 元。结果表明年度策略量级不依赖这一微小映射，但题目指定时段数值和逐槽费用仍需要统一定义，本文采用区间终点解释。

7.5 计算规模与结果可追溯性

每天的规划域最多为 144 槽，随着日内时间推进逐步缩短。每槽包含合同、闲置合同、光伏利用、充电、放电、应急与 SOC 等连续变量，约束数量随剩余槽数线性增长；所有目标和约束均为线性形式，可由通用线性规划器稳定求解。残差聚类的数据矩阵最多含 60 个历史日期、每行 288 维，且簇数不超过 9，因此其规模远小于把全年每个 10 分钟误差都展开成场景树的做法。该结构使 365 日连续模拟和 54 组实验在普通计算机

上可完成，同时保留逐槽检查的可能。

为了使结果可追溯，每个策略同时保留三层输出：第一层是逐槽合同、能量流和 SOC，用于检查式(2)、式(3)及指定时段；第二层是每日合同量、现金费用、应急量、闲置合同和弃光，用于代表日与时间序列分析；第三层是 334 日总量及实验参数，用于跨策略比较。论文中的所有汇总均从前一层按日期和时槽聚合，而不是在正文中手工修改。题目指定的五份工作簿与同一逐槽结果共用数据源，所以论文数值、代表日表和 Excel 输出保持同一口径。

随机性只出现在残差聚类的初始中心，随机种子固定为 2026，并使用 10 次初始化选择较小簇内平方和的结果。经验分位数、相似日权重、价格中位数、LP 和真实执行均为确定性步骤。固定种子并不说明模型没有统计不确定性，它只确保相同输入和软件环境下不会因聚类初始化改变合同轨迹。真正的外部不确定性仍来自未来负荷、光伏和价格，已通过滚动历史、分位参数与应急购电显式暴露。

结果解释遵循“数值可行、因果可用、外部有效性待检验”的层次。逐槽残差接近机器精度证明计算结果满足本文方程；未来扰动不影响当前决策证明实现遵守给定信息边界；54 组实验说明主要参数变化下结论如何移动。但这些证据不能自动证明另一年份、另一微网或另一合同条款下仍有同样节约率。工程迁移时应先复现输入映射和结算，再用滚动样本外数据重新校准历史窗口与保守度。

7.6 模型优点与局限

本文方法具有四点优点：第一，信息集明确，预测与执行均可通过未来扰动验证；第二，物理层完全线性，所有 10 分钟结果可逐槽复核；第三，联合残差在压缩前同时包含负荷和光伏误差，较单独安全系数更贴近净负荷风险；第四，两种结算、多个更新集合和设备参数均有真实重算或同策略重计费结果。

局限也十分明确。其一，边际分位压缩会丢失跨时段联合概率，不能提供机会约束保证。其二，相似日和历史中位数属于低参数预测，可能无法捕捉节假日、极端天气或结构突变。其三，真实执行采用优先级规则而非全年完美信息优化，设备参数的实际费用可能非单调。其四，储能老化、备用、配网潮流和需求响应不在题设数据中，本文没有虚构这些成本；若用于工程实施，应加入退化成本、网络约束和滚动终端集合。其五，2 小时方案是由已发布预报与观测构成的合成短临结果，不能替代对真实高频预报服务的独立评估。

8 结论与实施建议

本文用统一的 10 分钟能量平衡和储能状态方程连接四个问题，并将“当时可得信息”作为滚动决策的硬边界。主要结论如下。

第一，典型日确定性优化得到购电 59482.6982 kWh、费用 35126.9489 元。在 6000 kWh 循环 SOC 约束下，储能通过低价充电和高价放电完成能量转移，逐槽平衡残差低于 10^{-12} kWh。这一结果给出了后续全年模型的物理与量纲基线。

第二，在固定价格且不能日内调账时，相似日预测、联合残差聚类 and 边际分位轨迹共同构成因果日前策略。334 日现金费用为 1876.42 万元，应急购电 57.81 万 kWh。相

较点预测和典型日，保守轨迹显著降低应急费用；但经验 90% 区间实际覆盖率约 83%，说明该策略是风险启发式而非概率保证。

第三，在 0、6、12、18 时接收光伏预报并调整未执行合同时，费用降至 1627.97 万元，应急购电降至 11.55 万 kWh。相对问题二节约 248.45 万元，但该差异同时来自新信息和滚动规则。相同 0 时信息下，2 小时因果合成更新比 6 小时更新节约 70.83 万元；仅当额外更新成本低于约 265.10 元/次，且主结算解释成立时，增加频率才有正净收益。

第四，变动电价下应将未来价格也视为未知量。历史同槽中位数与当日已实现偏差构成的预测策略，在有调账时费用为 1673.51 万元，应急购电 11.59 万 kWh；相对无调账节约 251.94 万元。2 小时方案相对 6 小时方案节约 70.16 万元，对应更新成本阈值约 262.59 元/次。若采用替代结算解释，收益显著收缩，实际实施前必须先确认合同下调条款。

综合而言，建议把 6 小时官方发布节奏作为稳健基线；在数据接入稳定、主结算明确且单次增量成本低于约 260 元时，可试行 2 小时合成短临更新。上线时应同时监控合同闲置、应急量、弃光和 SOC 边界，而不能只用总费用判断。若后续获得多年份天气、节假日信息和真实高频预报，应采用滚动样本外检验重新选择 α 、历史窗口及更新频率，并把储能衰减和配电网约束纳入模型。

实施顺序上，可先将本文的 6 小时策略部署为影子调度，只记录建议合同而不直接执行，逐日核对预测误差、结算条款与 SOC 轨迹；确认数据时标和费用口径后，再逐步开放合同调整。2 小时合成更新应设置收益阈值和异常回退机制：当数据缺失、通信延迟或估计收益不足以覆盖更新成本时，自动退回最近一次官方发布方案。这样既保留滚动信息的经济价值，也避免把样本内最优频率机械固化为长期规则。

AI 工具使用声明

本文参考稿在资料整理、代码检查、结果表述、LaTeX 排版和语言润色阶段使用了人工智能辅助工具。核心模型采用的变量、约束、因果信息边界、实验口径及最终是否采纳，均需由参赛队伍结合题目和原始附件逐项判断。人工智能生成内容可能存在错误，本参考稿不代表队伍已经完成人工复核，也不能未经理解、核验和改写直接提交。所用工具、主要用途、交互示例、采纳范围与待人工核验事项另见支撑材料中的《AI 工具使用详情》。

参考文献

- [1] Jiefeng Hu, Yinghao Shan, Josep M. Guerrero, Adrian Ioinovici, Ka Wing Chan, and Jose Rodriguez. Model predictive control of microgrids – an overview. *Renewable and Sustainable Energy Reviews*, 136:110422, 2021.
- [2] Hao Liang and Weihua Zhuang. Stochastic modeling and optimization in a microgrid: A survey. *Energies*, 7(4):2027–2050, 2014.

- [3] Stefano Raimondi Cominesi, Marcello Farina, Luca Giulioni, Bruno Picasso, and Riccardo Scattolini. A two-layer stochastic model predictive control scheme for microgrids. *IEEE Transactions on Control Systems Technology*, 26(1):1–13, 2018.
- [4] Javier Silvente, Georgios M. Kopanos, and Antonio Espuña. A rolling horizon stochastic programming framework for the energy supply and demand management in microgrids. In *Computer Aided Chemical Engineering*, pages 2321–2326. 2015.
- [5] Huiru Zhao, Hao Lu, Bingkang Li, Xuejie Wang, Shiyang Zhang, and Yuwei Wang. Stochastic optimization of microgrid participating day-ahead market operation strategy with consideration of energy storage system and demand response. *Energies*, 13(5):1255, 2020.

A 代表日结果与题目指定表

本附录汇总四个代表日在四种年度主策略下的关键量。题目要求的五份完整工作簿与全部 10 分钟明细同时保存在支撑材料中；其表头保持题面格式。表8中的费用为当天真实执行后的现金费用，不是 0 时计划费用。

表 8 四个代表日的策略汇总

方案	日期	初始合同/kWh	最终合同/kWh	费用/元	应急/kWh
q2	03-20	75035.09	75035.09	46605.58	105.70
q2	06-21	72783.62	72783.62	44633.55	0.00
q2	09-23	70012.54	70012.54	51590.87	1188.95
q2	12-21	94716.85	94716.85	63120.31	382.23
q3	03-20	71214.48	67237.38	43146.11	53.05
q3	06-21	67214.74	64689.09	41572.47	0.00
q3	09-23	80713.74	77815.74	50639.84	0.00
q3	12-21	93486.19	93486.70	65731.59	979.74
q4 无调账	03-20	75035.10	75035.10	47419.90	56.96
q4 无调账	06-21	72896.92	72896.92	36993.89	0.00
q4 无调账	09-23	70077.68	70077.68	53101.80	1183.99
q4 无调账	12-21	94363.54	94363.54	74025.45	382.23
q4 有调账	03-20	71187.05	67231.66	43995.96	48.51
q4 有调账	06-21	67327.32	64812.17	35139.40	0.00
q4 有调账	09-23	80713.74	77818.09	53065.75	0.00
q4 有调账	12-21	93135.12	93169.88	77842.31	978.68

题目指定的购电表从每个代表日分别提取 10:00–10:10、12:00–12:10、14:00–14:10、16:00–16:10、18:00–18:10 和 20:00–20:10 六个自然时槽；无调账方案只使用初始合同，有调账方案同时保留初始与最终合同。指定储能表将 144 槽聚合为 0–4、4–8、8–12、12–16、

16–20、20–24 时六段，并列 0 时与 24 时 SOC。应急表只合并连续非零时槽，没有应急购电的日期标记为“无”。这些规则保证聚合结果与逐槽总量守恒。

为便于在正文外快速核对，表9给出代表日首末 SOC。全年模型没有每日循环约束，因而多数代表日首末 SOC 不同；恰好相同的情况来自策略在该日运行到同一边界，而非人为重置。

表 9 代表日首末 SOC (kWh)

日期	指标	q2	q3	q4 无调账	q4 有调账
03-20	0 时/24 时	10800/10774.98	10800/10800	10800/10774.98	10800/10800
06-21	0 时/24 时	10800/10800	10800/10800	10800/10800	10800/10800
09-23	0 时/24 时	10800/10800	10800/10800	10800/10800	10800/10800
12-21	0 时/24 时	10800/10710.17	10800/10756.15	10800/10702.85	10800/10751.81

配套文件为 **result1.xlsx**、**result2.xlsx**、**result3.xlsx**、**result4-2.xlsx** 和 **result4-3.xlsx**。其中问题二和问题四无调账方案的初始合同与最终合同相同；问题三和问题四有调账方案的最终合同由 0、6、12、18 时最后一次有效更新共同决定。

B 核心算法源码

下面给出负责数据读取、预测、场景压缩、线性规划、实际执行、价格预测和四题单日求解的核心模块。年度驱动、实验驱动、结果表生成和绘图程序作为独立源文件随支撑材料提供，以便运行和检查。本附录源码按行自动换行，长标识符的视觉折行不改变文件内容。

```

1 """2026 CUMCM C problem: shared data, forecasting, LP, and causal simulation core."""
2
3 from __future__ import annotations
4
5 from dataclasses import dataclass
6 import os
7 from pathlib import Path
8 from typing import Any
9
10 import numpy as np
11 import pandas as pd
12 from scipy.optimize import linprog
13 from scipy.sparse import csr_matrix, lil_matrix
14 os.environ.setdefault("LOKY_MAX_CPU_COUNT", "4")
15
16 from sklearn.cluster import KMeans
17
18
19 DT_HOURS = 1.0 / 6.0
20 N_SLOTS = 144
21 ETA_C = 0.90
22 ETA_D = 0.90
23 SOC_MIN = 1_200.0
24 SOC_MAX = 10_800.0
25 SOC_INITIAL = 6_000.0
26 POWER_LIMIT_KW = 5_000.0

```

```

27 ENERGY_LIMIT = POWER_LIMIT_KW * DT_HOURS
28 SEED = 2026
29 DEFAULT_ALPHA = 0.80
30 ISSUES = (0, 6, 12, 18)
31
32
33 @dataclass(frozen=True)
34 class Battery:
35     capacity_kwh: float = 12000.0
36     power_kw: float = POWER_LIMIT_KW
37     eta_c: float = ETA_C
38     eta_d: float = ETA_D
39
40     @property
41     def minimum(self) -> float:
42         return 0.1 * self.capacity_kwh
43
44     @property
45     def maximum(self) -> float:
46         return 0.9 * self.capacity_kwh
47
48     @property
49     def initial(self) -> float:
50         return 0.5 * self.capacity_kwh
51
52     @property
53     def slot_limit(self) -> float:
54         return self.power_kw * DT_HOURS
55
56
57 DEFAULT_BATTERY = Battery()
58
59
60 @dataclass(frozen=True)
61 class Inputs:
62     dates: pd.DatetimeIndex
63     typical_time: list[Any]
64     typical_price: np.ndarray
65     typical_load_kw: np.ndarray
66     typical_pv_kw: np.ndarray
67     actual_load_kw: np.ndarray
68     actual_pv_kw: np.ndarray
69     variable_price: np.ndarray
70     pv_forecast_hourly: np.ndarray
71
72
73 def load_inputs(project_root: str | Path) -> Inputs:
74     root = Path(project_root)
75     attachment = root / "data" / "附件"
76
77     a1 = pd.read_excel(attachment / "附件 1.xlsx", sheet_name=0, header=0)
78     if a1.shape != (N_SLOTS, 4):
79         raise ValueError(f"附件 1 维度异常: {a1.shape}")
80
81     load_df = pd.read_excel(attachment / "附件2.xlsx", sheet_name="小区负载", index_col=0, header=0)
82     pv_df = pd.read_excel(attachment / "附件2.xlsx", sheet_name="光伏发电实际功率", index_col=0,
83                           header=0)
84     price_df = pd.read_excel(attachment / "附件 4.xlsx", sheet_name=0, index_col=0, header=0)
85     def clock_minute(value):
86         if hasattr(value, "hour"):
87             minute = value.hour * 60 + value.minute
88         else:
89             clock = str(value).split("+")[0].split(":")
90             minute = int(clock[0]) * 60 + int(clock[1])

```

```

90         return 1440 if minute == 0 else minute
91     expected_slots = list(range(10, 1441, 10))
92     for labels in (a1.iloc[:, 0], load_df.columns, pv_df.columns, price_df.columns):
93         if [clock_minute(v) for v in labels] != expected_slots:
94             raise ValueError(" 附件时间列不是有序的 00:10 至 24:00 端点")
95     dates = pd.DatetimeIndex(pd.to_datetime(load_df.index))
96     if load_df.shape != (365, N_SLOTS) or pv_df.shape != load_df.shape:
97         raise ValueError(" 附件 2 应为 365×144 的负荷和光伏矩阵")
98     if price_df.shape != load_df.shape:
99         raise ValueError(" 附件 4 应与附件 2 严格同形")
100    if not dates.equals(pd.DatetimeIndex(pd.to_datetime(pv_df.index))):
101        raise ValueError(" 附件 2 两个工作表日期不一致")
102    if not dates.equals(pd.DatetimeIndex(pd.to_datetime(price_df.index))):
103        raise ValueError(" 附件 4 日期与附件 2 不一致")
104
105    a3 = pd.read_excel(attachment / " 附件 3.xlsx", sheet_name=0, header=0)
106    if a3.shape != (1460, 26) or list(a3.columns[2:]) != [f" 预报 {h} 小时" for h in range(1, 25)]:
107        raise ValueError(" 附件 3 维度或提前期表头异常")
108    forecast_dates = pd.to_datetime(a3.iloc[:, 0].replace("", np.nan).ffill())
109    forecast = np.full((len(dates), len(ISSUES), 24), np.nan, dtype=float)
110    date_pos = {d.normalize(): i for i, d in enumerate(dates)}
111    issue_pos = {h: i for i, h in enumerate(ISSUES)}
112    for row_index, row in enumerate(a3.itertuples(index=False, name=None)):
113        d = pd.Timestamp(forecast_dates.iloc[row_index]).normalize()
114        issue_value = row[1]
115        issue_hour = int(issue_value.hour) if hasattr(issue_value, "hour") else
116        int(str(issue_value).split(":")[0])
117        if d not in date_pos or issue_hour not in issue_pos:
118            raise ValueError(" 附件 3 含未定义日期或发布时刻")
119        target = forecast[date_pos[d], issue_pos[issue_hour]]
120        if np.isfinite(target).any():
121            raise ValueError(" 附件 3 日期与发布时刻重复")
122        target[:] = np.asarray(row[2:26], dtype=float)
123    if np.isnan(forecast).any():
124        raise ValueError(" 附件 3 存在未能映射的日期、发布时刻或预测值")
125
126    result = Inputs(
127        dates=dates,
128        typical_time=a1.iloc[:, 0].tolist(),
129        typical_price=a1.iloc[:, 1].to_numpy(dtype=float),
130        typical_load_kw=a1.iloc[:, 2].to_numpy(dtype=float),
131        typical_pv_kw=a1.iloc[:, 3].to_numpy(dtype=float),
132        actual_load_kw=load_df.to_numpy(dtype=float),
133        actual_pv_kw=pv_df.to_numpy(dtype=float),
134        variable_price=price_df.to_numpy(dtype=float),
135        pv_forecast_hourly=forecast,
136    )
137    _validate_inputs(result)
138    return result
139
140    def _validate_inputs(data: Inputs) -> None:
141        arrays = (
142            data.typical_price,
143            data.typical_load_kw,
144            data.typical_pv_kw,
145            data.actual_load_kw,
146            data.actual_pv_kw,
147            data.variable_price,
148            data.pv_forecast_hourly,
149        )
150        if any(not np.isfinite(a).all() for a in arrays):
151            raise ValueError(" 输入包含 NaN 或无穷值")
152        if any((a < 0).any() for a in arrays):

```

```

153         raise ValueError(" 输入包含负的电价、负荷或光伏值")
154     expected = pd.date_range("2025-01-01", "2025-12-31", freq="D")
155     if not data.dates.equals(expected):
156         raise ValueError(" 附件日期并非 2025 完整连续自然年")
157
158
159 def build_analog_forecasts(
160     actual_kw: np.ndarray,
161     typical_kw: np.ndarray,
162     *,
163     weekday_weight: bool,
164     max_history: int = 90,
165     decay_days: float = 60.0,
166 ) -> tuple[np.ndarray, np.ndarray]:
167     """Strict rolling-origin analog forecast and its realized residuals."""
168     n_days, n_slots = actual_kw.shape
169     pred = np.empty_like(actual_kw, dtype=float)
170     for d in range(n_days):
171         start = max(0, d - max_history)
172         idx = np.arange(start, d)
173         if idx.size == 0:
174             pred[d] = typical_kw
175             continue
176         ages = d - idx
177         weights = np.exp(-ages / decay_days)
178         if weekday_weight:
179             weights *= np.where(idx % 7 == d % 7, 2.0, 1.0)
180         hist_mean = np.average(actual_kw[idx], axis=0, weights=weights)
181         shrink = min(1.0, idx.size / 21.0)
182         pred[d] = shrink * hist_mean + (1.0 - shrink) * typical_kw
183     pred = np.maximum(pred, 0.0)
184     return pred, actual_kw - pred
185
186
187 def _cluster_residuals(
188     load_residual_kw: np.ndarray,
189     pv_residual_kw: np.ndarray,
190     *,
191     max_scenarios: int = 9,
192 ) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
193     n = load_residual_kw.shape[0]
194     if n == 0:
195         zeros = np.zeros((1, load_residual_kw.shape[1]), dtype=float)
196         return zeros, zeros.copy(), np.ones(1)
197     raw = np.concatenate([load_residual_kw, pv_residual_kw], axis=1)
198     scale = np.std(raw, axis=0)
199     scale[scale < 1e-9] = 1.0
200     k = min(max_scenarios, n, np.unique(raw / scale, axis=0).shape[0])
201     if k == 1:
202         labels = np.zeros(n, dtype=int)
203     else:
204         labels = KMeans(n_clusters=k, random_state=SEED, n_init=10).fit_predict(raw / scale)
205     nonempty = np.unique(labels)
206     centers_l = np.stack([load_residual_kw[labels == i].mean(axis=0) for i in nonempty])
207     centers_v = np.stack([pv_residual_kw[labels == i].mean(axis=0) for i in nonempty])
208     weights = np.asarray([(labels == i).mean() for i in nonempty], dtype=float)
209     return centers_l, centers_v, weights
210
211
212 def weighted_quantile_columns(values: np.ndarray, weights: np.ndarray, quantile: float) ->
    np.ndarray:
213     if values.ndim != 2 or values.shape[0] != weights.size:
214         raise ValueError(" 分位数输入维度不一致")

```

```

215     if not 0 <= quantile <= 1 or not np.isfinite(values).all() or not np.isfinite(weights).all() or
216         np.any(weights < 0) or weights.sum() <= 0:
217         raise ValueError(" 分位数或概率非法")
218     result = np.empty(values.shape[1], dtype=float)
219     for t in range(values.shape[1]):
220         order = np.argsort(values[:, t], kind="mergesort")
221         v = values[order, t]
222         w = weights[order]
223         cdf = np.cumsum(w) / np.sum(w)
224         result[t] = v[min(np.searchsorted(cdf, quantile, side="left"), len(v) - 1)]
225     return result
226
227 def safe_analog_trajectory(
228     day: int,
229     load_point_kw: np.ndarray,
230     pv_point_kw: np.ndarray,
231     load_residual_kw: np.ndarray,
232     pv_residual_kw: np.ndarray,
233     *,
234     alpha: float = DEFAULT_ALPHA,
235     history_days: int = 60,
236     max_scenarios: int = 9,
237 ) -> tuple[np.ndarray, np.ndarray, dict[str, Any]]:
238     start = max(0, day - history_days)
239     c_l, c_v, weights = _cluster_residuais(
240         load_residual_kw[start:day], pv_residual_kw[start:day], max_scenarios=max_scenarios
241     )
242     l_scen = np.maximum(0.0, load_point_kw[None, :] + c_l)
243     v_scen = np.maximum(0.0, pv_point_kw[None, :] + c_v)
244     net_q = weighted_quantile_columns(l_scen - v_scen, weights, alpha)
245     pv_low = weighted_quantile_columns(v_scen, weights, 1.0 - alpha)
246     safe_pv = np.maximum.reduce([pv_low, -net_q, np.zeros_like(net_q)])
247     safe_load = safe_pv + net_q
248     return safe_load, safe_pv, {
249         "alpha": alpha,
250         "history_start": start,
251         "history_end_exclusive": day,
252         "scenario_count": int(weights.size),
253         "scenario_weights": weights.tolist(),
254     }
255
256
257 def _block_slices(n: int, adjusted: bool) -> dict[str, slice]:
258     pos = 0
259     blocks: dict[str, slice] = {}
260     for name, size in (("grid", n), ("charge", n), ("discharge", n), ("soc", n + 1), ("pv_used",
261         n)):
262         blocks[name] = slice(pos, pos + size)
263         pos += size
264     if adjusted:
265         blocks["up"] = slice(pos, pos + n)
266         pos += n
267         blocks["down"] = slice(pos, pos + n)
268         pos += n
269     blocks["all"] = slice(0, pos)
270     return blocks
271
272 def solve_schedule(
273     load_kw: np.ndarray,
274     pv_kw: np.ndarray,
275     price: np.ndarray,

```

```

276     soc0: float,
277     *,
278     terminal_cycle: bool = False,
279     terminal_value: float = 0.0,
280     base_plan_kwh: np.ndarray | None = None,
281     battery: Battery = DEFAULT_BATTERY,
282 ) -> dict[str, Any]:
283     """Solve a deterministic LP for an initial or adjusted contract trajectory."""
284     load_kw = np.asarray(load_kw, dtype=float)
285     pv_kw = np.asarray(pv_kw, dtype=float)
286     price = np.asarray(price, dtype=float)
287     n = load_kw.size
288     if pv_kw.shape != (n,) or price.shape != (n,):
289         raise ValueError(" 负荷、光伏、电价长度不一致")
290     if not (battery.minimum - 1e-9 <= soc0 <= battery.maximum + 1e-9):
291         raise ValueError(f" 初始 SOC 越界: {soc0}")
292     adjusted = base_plan_kwh is not None
293     if adjusted:
294         base_plan_kwh = np.asarray(base_plan_kwh, dtype=float)
295         if base_plan_kwh.shape != (n,):
296             raise ValueError(" 原计划与调整时域长度不一致")
297     b = _block_slices(n, adjusted)
298     m = b["all"].stop
299     c = np.zeros(m, dtype=float)
300     if adjusted:
301         c[b["up"]] = 1.5 * price
302         c[b["down"]] = -0.5 * price
303         constant_cost = float(np.dot(price, base_plan_kwh))
304     else:
305         c[b["grid"]] = price
306         constant_cost = 0.0
307     c[b["soc"].stop - 1] = -float(terminal_value)
308
309     rows = 2 * n + (n if adjusted else 0)
310     aeq = lil_matrix((rows, m), dtype=float)
311     beq = np.zeros(rows, dtype=float)
312     load_energy = load_kw * DT_HOURS
313     pv_energy = pv_kw * DT_HOURS
314     for t in range(n):
315         aeq[t, b["grid"].start + t] = 1.0
316         aeq[t, b["charge"].start + t] = -1.0
317         aeq[t, b["discharge"].start + t] = 1.0
318         aeq[t, b["pv_used"].start + t] = 1.0
319         beq[t] = load_energy[t]
320
321         row = n + t
322         aeq[row, b["soc"].start + t] = -1.0
323         aeq[row, b["soc"].start + t + 1] = 1.0
324         aeq[row, b["charge"].start + t] = -battery.eta_c
325         aeq[row, b["discharge"].start + t] = 1.0 / battery.eta_d
326     if adjusted:
327         row = 2 * n + t
328         aeq[row, b["grid"].start + t] = 1.0
329         aeq[row, b["up"].start + t] = -1.0
330         aeq[row, b["down"].start + t] = 1.0
331         beq[row] = base_plan_kwh[t]
332
333     bounds: list[tuple[float | None, float | None]] = []
334     bounds.extend([(0.0, None)] * n)
335     bounds.extend([(0.0, battery.slot_limit)] * n)
336     bounds.extend([(0.0, battery.slot_limit)] * n)
337     soc_bounds = [(battery.minimum, battery.maximum)] * (n + 1)
338     soc_bounds[0] = (soc0, soc0)
339     if terminal_cycle:

```

```

340     soc_bounds[-1] = (soc0, soc0)
341     bounds.extend(soc_bounds)
342     bounds.extend([(0.0, float(v)) for v in pv_energy])
343     if adjusted:
344         bounds.extend([(0.0, None)] * n)
345         bounds.extend([(0.0, float(v)) for v in base_plan_kwh])
346
347     res = linprog(c, A_eq=aeq.tocsr(), b_eq=beq, bounds=bounds, method="highs")
348     if not res.success:
349         raise RuntimeError(f"HiGHS 求解失败: status={res.status}, {res.message}")
350
351     economic_optimum = float(res.fun)
352     tie = np.zeros(m, dtype=float)
353     tie[b["charge"]] = 1.0
354     tie[b["discharge"]] = 1.0
355     aub = csr_matrix(c.reshape(1, -1))
356     tol = 1e-8 * max(1.0, abs(economic_optimum))
357     res2 = linprog(
358         tie,
359         A_ub=aub,
360         b_ub=np.asarray([economic_optimum + tol]),
361         A_eq=aeq.tocsr(),
362         b_eq=beq,
363         bounds=bounds,
364         method="highs",
365     )
366     if not res2.success:
367         raise RuntimeError(f"吞吐次级 LP 失败: {res2.status}, {res2.message}")
368     x = res2.x
369     balance_residual = aeq.tocsr() @ x - beq
370     result = {
371         "grid_kwh": x[b["grid"]].copy(),
372         "charge_kwh": x[b["charge"]].copy(),
373         "discharge_kwh": x[b["discharge"]].copy(),
374         "soc_kwh": x[b["soc"]].copy(),
375         "pv_used_kwh": x[b["pv_used"]].copy(),
376         "solver_objective": float(np.dot(c, x) + constant_cost + terminal_value * battery.minimum),
377         "cash_contract_cost": float(
378             np.dot(price, x[b["grid"]])
379             if not adjusted
380             else constant_cost
381             + np.dot(1.5 * price, x[b["up"]])
382             - np.dot(0.5 * price, x[b["down"]])
383         ),
384         "terminal_value_credit": float(terminal_value * (x[b["soc"].stop - 1] - battery.minimum)),
385         "max_eq_residual": float(np.max(np.abs(balance_residual))),
386         "simultaneous_flow_max": float(np.max(np.minimum(x[b["charge"]], x[b["discharge"]]))),
387         "status": "optimal",
388     }
389     if adjusted:
390         result["up_kwh"] = x[b["up"]].copy()
391         result["down_kwh"] = x[b["down"]].copy()
392     return result
393
394
395 def causal_dispatch(
396     contract_kwh: np.ndarray,
397     actual_load_kw: np.ndarray,
398     actual_pv_kw: np.ndarray,
399     price: np.ndarray,
400     soc0: float,
401     *,
402     base_plan_kwh: np.ndarray | None = None,
403     battery: Battery = DEFAULT_BATTERY,

```

```

404     emergency_multiplier: float = 5.0,
405 ) -> dict[str, Any]:
406     """Greedy feasible controller using current observation only."""
407     q = np.asarray(contract_kwh, dtype=float)
408     load_e = np.asarray(actual_load_kw, dtype=float) * DT_HOURS
409     pv_e = np.asarray(actual_pv_kw, dtype=float) * DT_HOURS
410     price = np.asarray(price, dtype=float)
411     n = q.size
412     if load_e.shape != (n,) or pv_e.shape != (n,) or price.shape != (n,):
413         raise ValueError(" 因果控制输入长度不一致")
414     charge = np.zeros(n)
415     discharge = np.zeros(n)
416     emergency = np.zeros(n)
417     unused = np.zeros(n)
418     curtailed = np.zeros(n)
419     pv_used = np.zeros(n)
420     soc = np.empty(n + 1)
421     soc[0] = soc0
422     for t in range(n):
423         available = q[t] + pv_e[t] - load_e[t]
424         if available >= 0.0:
425             charge[t] = min(available, battery.slot_limit, max(0.0, (battery.maximum - soc[t]) /
426                 battery.eta_c))
427             surplus = max(0.0, available - charge[t])
428             unused[t] = min(q[t], surplus)
429             curtailed[t] = max(0.0, surplus - unused[t])
430             pv_used[t] = pv_e[t] - curtailed[t]
431             soc[t + 1] = soc[t] + battery.eta_c * charge[t]
432         else:
433             need = -available
434             discharge[t] = min(need, battery.slot_limit, max(0.0, (soc[t] - battery.minimum) *
435                 battery.eta_d))
436             emergency[t] = max(0.0, need - discharge[t])
437             pv_used[t] = pv_e[t]
438             soc[t + 1] = soc[t] - discharge[t] / battery.eta_d
439     residual = q - unused + pv_used + discharge + emergency - load_e - charge
440     if base_plan_kwh is None:
441         settlement = price * q
442         alt_settlement = settlement.copy()
443     else:
444         p = np.asarray(base_plan_kwh, dtype=float)
445         if p.shape != (n,):
446             raise ValueError(" 原计划与最终合同长度不一致")
447         settlement = price * np.minimum(p, q) + 1.5 * price * np.maximum(q - p, 0.0) + 0.5 * price
448         * np.maximum(p - q, 0.0)
449         alt_settlement = price * p + 1.5 * price * np.maximum(q - p, 0.0) + 0.5 * price *
450         np.maximum(p - q, 0.0)
451     emergency_cost = emergency_multiplier * price * emergency
452     return {
453         "charge_kwh": charge,
454         "discharge_kwh": discharge,
455         "soc_kwh": soc,
456         "emergency_kwh": emergency,
457         "unused_contract_kwh": unused,
458         "pv_curtailed_kwh": curtailed,
459         "pv_used_kwh": pv_used,
460         "settlement_cost": settlement,
461         "alternative_settlement_cost": alt_settlement,
462         "emergency_cost": emergency_cost,
463         "cash_cost": settlement + emergency_cost,
464         "alternative_cash_cost": alt_settlement + emergency_cost,
465         "max_balance_residual": float(np.max(np.abs(residual))),
466     }

```

```

463
464
465 def terminal_value(price: np.ndarray, battery: Battery = DEFAULT_BATTERY) -> float:
466     return float(battery.eta_d * np.median(np.asarray(price, dtype=float)))
467
468
469 def predict_price(data: Inputs, day: int, issue_hour: int = 0) -> np.ndarray:
470     """Only historical days and the already-realized prefix are readable."""
471     base = np.median(data.variable_price[max(0, day - 30):day], axis=0) if day else
472     data.typical_price.copy()
473     start = issue_hour * 6
474     if start:
475         observed = slice(max(0, start - 18), start)
476         bias = float(np.mean(data.variable_price[day, observed] - base[observed]))
477         tau = np.arange(1, N_SLOTS - start + 1) / 6.0
478         return np.maximum(0.0001, base[start:] + bias * np.exp(-tau / 3.0))
479     return np.maximum(0.0001, base)
480
481 def run_q1(data: Inputs, battery: Battery = DEFAULT_BATTERY) -> dict[str, Any]:
482     return solve_schedule(
483         data.typical_load_kw,
484         data.typical_pv_kw,
485         data.typical_price,
486         battery.initial,
487         terminal_cycle=True,
488         battery=battery,
489     )
490
491
492 def run_q2_day(
493     data: Inputs,
494     day: int,
495     soc0: float,
496     load_point: np.ndarray,
497     pv_point: np.ndarray,
498     load_residual: np.ndarray,
499     pv_residual: np.ndarray,
500     *,
501     variable_price: bool,
502     alpha: float = DEFAULT_ALPHA,
503     safe_pair: tuple[np.ndarray, np.ndarray, dict[str, Any]] | None = None,
504     battery: Battery = DEFAULT_BATTERY,
505     emergency_multiplier: float = 5.0,
506     history_days: int = 60,
507     max_scenarios: int = 9,
508 ) -> dict[str, Any]:
509     price = predict_price(data, day) if variable_price else data.typical_price
510     settlement_price = data.variable_price[day] if variable_price else data.typical_price
511     if safe_pair is None:
512         safe_l, safe_v, meta = safe_analog_trajectory(
513             day, load_point[day], pv_point[day], load_residual, pv_residual, alpha=alpha,
514             history_days=history_days, max_scenarios=max_scenarios
515         )
516     else:
517         safe_l, safe_v, meta = safe_pair
518     plan = solve_schedule(
519         safe_l,
520         safe_v,
521         price,
522         soc0,
523         terminal_value=terminal_value(price, battery),
524         battery=battery,

```

```

524     )
525     actual = causal_dispatch(
526         plan["grid_kwh"], data.actual_load_kw[day], data.actual_pv_kw[day], settlement_price, soc0,
527         battery=battery, emergency_multiplier=emergency_multiplier
528     )
529     return {"plan": plan, "actual": actual, "forecast_meta": meta, "safe_load_kw": safe_l,
530            "safe_pv_kw": safe_v, "planning_price": price}
531
532 def official_pv_residuals(data: Inputs) -> np.ndarray:
533     """Actual power at each forecasted clock hour minus that point forecast."""
534     n_days = len(data.dates)
535     flat = data.actual_pv_kw.reshape(-1)
536     residual = np.full_like(data.pv_forecast_hourly, np.nan, dtype=float)
537     for d in range(n_days):
538         for i, issue in enumerate(ISSUES):
539             start = d * N_SLOTS + issue * 6
540             for h in range(24):
541                 # Column h is the point forecast at issue+(h+1) hours. The
542                 # 10-minute source rows are interval endpoints, hence -1.
543                 endpoint = start + (h + 1) * 6 - 1
544                 if endpoint < flat.size:
545                     residual[d, i, h] = flat[endpoint] - data.pv_forecast_hourly[d, i, h]
546     return residual
547
548 def interpolate_hourly_points(anchor: float, hourly_points: np.ndarray, n_slots: int) ->
549     np.ndarray:
550     """Linearly interpolate clock-hour point values onto 10-minute endpoints."""
551     values = np.concatenate([float(anchor)], np.asarray(hourly_points, dtype=float)))
552     x_hour = np.arange(values.size, dtype=float)
553     x_slot = np.arange(1, n_slots + 1, dtype=float) / 6.0
554     return np.interp(x_slot, x_hour, values)
555
556 def safe_update_trajectory(
557     data: Inputs,
558     day: int,
559     issue_index: int,
560     load_point_day: np.ndarray,
561     load_residual: np.ndarray,
562     pv_issue_residual: np.ndarray,
563     *,
564     alpha: float = DEFAULT_ALPHA,
565     history_days: int = 60,
566     max_scenarios: int = 9,
567 ) -> tuple[np.ndarray, np.ndarray, dict[str, Any]]:
568     issue = ISSUES[issue_index]
569     start_slot = issue * 6
570     n = N_SLOTS - start_slot
571     hours = int(np.ceil(n / 6))
572     raw = data.pv_forecast_hourly[day, issue_index, :hours]
573     hist_start = max(0, day - history_days)
574     hist = pv_issue_residual[hist_start:day, issue_index, :hours]
575     valid_rows = np.all(np.isfinite(hist), axis=1) if hist.size else np.zeros(0, dtype=bool)
576     hist = hist[valid_rows]
577     bias = np.nanmean(hist, axis=0) if hist.shape[0] else np.zeros(hours)
578     if issue > 0:
579         anchor = float(data.actual_pv_kw[day, start_slot - 1])
580     elif day > 0:
581         anchor = float(data.actual_pv_kw[day - 1, -1])
582     else:
583         anchor = 0.0

```

```

584     pv_point = interpolate_hourly_points(anchor, np.maximum(0.0, raw + bias), n)
585     l_point = load_point_day[start_slot:]
586
587     hist_ids = np.arange(hist_start, day)[valid_rows]
588     if hist_ids.size:
589         r_l = load_residual[hist_ids, start_slot:]
590         centered_hourly = hist - bias[None, :]
591         r_v = np.vstack(
592             [interpolate_hourly_points(0.0, row, n) for row in centered_hourly]
593         )
594     else:
595         r_l = np.empty((0, n))
596         r_v = np.empty((0, n))
597     c_l, c_v, weights = _cluster_residuals(r_l, r_v, max_scenarios=max_scenarios)
598     l_scen = np.maximum(0.0, l_point[None, :] + c_l)
599     v_scen = np.maximum(0.0, pv_point[None, :] + c_v)
600     net_q = weighted_quantile_columns(l_scen - v_scen, weights, alpha)
601     pv_low = weighted_quantile_columns(v_scen, weights, 1.0 - alpha)
602     safe_v = np.maximum.reduce([pv_low, -net_q, np.zeros_like(net_q)])
603     safe_l = safe_v + net_q
604     return safe_l, safe_v, {
605         "decision_day": int(day),
606         "issue_hour": int(issue),
607         "history_start": int(hist_start),
608         "history_end_exclusive": int(day),
609         "scenario_count": int(weights.size),
610         "alpha": float(alpha),
611         "scenario_weights": weights.tolist(),
612         "pv_point_kw": pv_point.tolist(),
613         "anchor_kw": anchor,
614         "bias_kw": bias.tolist(),
615     }
616
617
618 def nowcast_trajectory(
619     data: Inputs, day: int, event: int,
620     source: tuple[np.ndarray, np.ndarray, dict[str, Any]],
621 ) -> tuple[np.ndarray, np.ndarray, dict[str, Any]]:
622     """Synthetic nowcast based on the latest available official issue only."""
623     safe_l, safe_v, meta = source
624     issue = int(meta["issue_hour"])
625     offset = (event - issue) * 6
626     if offset < 0:
627         raise ValueError(" 不能使用未来发布的预报")
628     if not offset:
629         return source
630     point = np.asarray(meta["pv_point_kw"])
631     first = max(0, offset - 18)
632     realized = data.actual_pv_kw[day, issue * 6 + first:event * 6]
633     bias = float(np.mean(realized - point[first:offset])) if realized.size else 0.0
634     tau = np.arange(1, N_SLOTS - event * 6 + 1) / 6.0
635     revised = np.maximum(0.0, safe_v[offset:] + bias * np.exp(-tau / 3.0))
636     return safe_l[offset:], revised, {
637         **{k: v for k, v in meta.items() if k not in ("pv_point_kw", "bias_kw")},
638         "decision_hour": event, "synthetic_nowcast": True,
639         "observed_end_exclusive": event * 6, "nowcast_bias_kw": bias,
640     }
641
642
643 def run_q3_day(
644     data: Inputs,
645     day: int,
646     soc0: float,
647     load_point: np.ndarray,

```

```

648     load_residual: np.ndarray,
649     pv_issue_residual: np.ndarray,
650     *,
651     variable_price: bool,
652     alpha: float = DEFAULT_ALPHA,
653     safe_updates: list[tuple[np.ndarray, np.ndarray, dict[str, Any]]] | None = None,
654     events: tuple[int, ...] = ISSUES,
655     battery: Battery = DEFAULT_BATTERY,
656     emergency_multiplier: float = 5.0,
657     history_days: int = 60,
658     max_scenarios: int = 9,
659 ) -> dict[str, Any]:
660     if not events or events[0] != 0 or tuple(sorted(set(events))) != events or any(h < 0 or h >= 24
        or int(h) != h for h in events):
661         raise ValueError(" 更新时刻必须从 0 开始, 严格递增且在 [0,24) 内")
662     price = predict_price(data, day) if variable_price else data.typical_price
663     settlement_price = data.variable_price[day] if variable_price else data.typical_price
664     if safe_updates is None:
665         safe_l0, safe_v0, meta0 = safe_update_trajectory(
666             data, day, 0, load_point[day], load_residual, pv_issue_residual, alpha=alpha,
667             history_days=history_days, max_scenarios=max_scenarios
668         )
669     else:
670         safe_l0, safe_v0, meta0 = safe_updates[0]
671     initial = solve_schedule(
672         safe_l0,
673         safe_v0,
674         price,
675         soc0,
676         terminal_value=terminal_value(price, battery),
677         battery=battery,
678     )
679     p0 = initial["grid_kwh"]
680     final_contract = p0.copy()
681     actual_parts: dict[str, list[np.ndarray]] = {
682         key: []
683         for key in (
684             "charge_kwh",
685             "discharge_kwh",
686             "emergency_kwh",
687             "unused_contract_kwh",
688             "pv_curtailed_kwh",
689             "pv_used_kwh",
690             "settlement_cost",
691             "alternative_settlement_cost",
692             "emergency_cost",
693             "cash_cost",
694             "alternative_cash_cost",
695         )
696     }
697     soc_trace = [float(soc0)]
698     update_meta = [meta0]
699     update_solutions: list[dict[str, Any]] = []
700     current_soc = float(soc0)
701     for i, issue in enumerate(events):
702         start = issue * 6
703         end = events[i + 1] * 6 if i + 1 < len(events) else N_SLOTS
704         if i == 0:
705             contract_remaining = p0[start:]
706             update_solution = initial
707         else:
708             latest_index = max(j for j, h in enumerate(ISSUES) if h <= issue)

```

```

709         if safe_updates is None:
710             source = safe_update_trajectory(
711                 data, day, latest_index, load_point[day], load_residual, pv_issue_residual,
712                 alpha=alpha, history_days=history_days, max_scenarios=max_scenarios
713             )
714         else:
715             source = safe_updates[latest_index]
716             safe_l, safe_v, meta = nowcast_trajectory(data, day, issue, source)
717             update_meta.append(meta)
718             remaining_price = predict_price(data, day, issue) if variable_price else price[start:]
719             update_solution = solve_schedule(
720                 safe_l,
721                 safe_v,
722                 remaining_price,
723                 current_soc,
724                 terminal_value=terminal_value(remaining_price, battery),
725                 base_plan_kwh=p0[start:],
726                 battery=battery,
727             )
728             update_solution["planning_price"] = remaining_price
729             contract_remaining = update_solution["grid_kwh"]
730             update_solutions.append(update_solution)
731             segment_contract = contract_remaining[: end - start]
732             final_contract[start:end] = segment_contract
733             segment = causal_dispatch(
734                 segment_contract,
735                 data.actual_load_kw[day, start:end],
736                 data.actual_pv_kw[day, start:end],
737                 settlement_price[start:end],
738                 current_soc,
739                 base_plan_kwh=p0[start:end],
740                 battery=battery,
741                 emergency_multiplier=emergency_multiplier,
742             )
743             for key in actual_parts:
744                 actual_parts[key].append(segment[key])
745             soc_trace.extend(segment["soc_kwh"][1:].tolist())
746             current_soc = float(segment["soc_kwh"][-1])
747
748         actual = {key: np.concatenate(parts) for key, parts in actual_parts.items()}
749         actual["soc_kwh"] = np.asarray(soc_trace)
750         balance = final_contract - actual["unused_contract_kwh"] + actual["pv_used_kwh"] +
751         actual["discharge_kwh"] + actual["emergency_kwh"] - data.actual_load_kw[day] * DT_HOURS -
752         actual["charge_kwh"]
753         actual["max_balance_residual"] = float(np.max(np.abs(balance)))
754     return {
755         "initial_plan": initial,
756         "initial_plan_kwh": p0,
757         "final_contract_kwh": final_contract,
758         "actual": actual,
759         "update_meta": update_meta,
760         "update_solutions": update_solutions,
761         "planning_price": price,
762         "events": events,
763     }
764
765 def summarize_day(result: dict[str, Any], *, adjusted: bool) -> dict[str, float]:
766     actual = result["actual"]
767     plan = result["initial_plan_kwh"] if adjusted else result["plan"]["grid_kwh"]
768     contract = result["final_contract_kwh"] if adjusted else plan
769     return {
770         "plan_kwh": float(np.sum(plan)),

```

```

769     "final_contract_kwh": float(np.sum(contract)),
770     "cash_cost_yuan": float(np.sum(actual["cash_cost"])),
771     "alternative_cash_cost_yuan": float(np.sum(actual.get("alternative_cash_cost",
772     actual["cash_cost"]))),
773     "emergency_kwh": float(np.sum(actual["emergency_kwh"])),
774     "unused_contract_kwh": float(np.sum(actual["unused_contract_kwh"])),
775     "pv_curtailed_kwh": float(np.sum(actual["pv_curtailed_kwh"])),
776     "charge_kwh": float(np.sum(actual["charge_kwh"])),
777     "discharge_kwh": float(np.sum(actual["discharge_kwh"])),
778     "soc_start_kwh": float(actual["soc_kwh"][0]),
779     "soc_end_kwh": float(actual["soc_kwh"][-1]),
780     "soc_min_kwh": float(np.min(actual["soc_kwh"])),
781     "soc_max_kwh": float(np.max(actual["soc_kwh"])),
782     "max_balance_residual_kwh": float(actual["max_balance_residual"]),
783 }

```

C 支撑材料与运行说明

支撑材料包括以下内容：

- (1) 核心模型与执行程序：
microgrid_core.py、run_all.py;
evaluate_results.py;
- (2) 54 组实验、绘图与检查程序：
run_experiments.py、plot_all.py;
audit_results.py、audit_experiments.py;
- (3) 单元测试和统一入口：
test_core.py、test_evaluation.py;
smoke_test.py、reproduce.py;
- (4) 题目原始附件、五份结果工作簿、关键 CSV 汇总、12 幅论文图及可编辑 SVG 图;
- (5) 运行环境说明、依赖列表、模型参数说明和《AI 工具使用详情》。

在项目根目录中使用 Python 3.14 或兼容版本，安装 NumPy、pandas、SciPy、scikit-learn、openpyxl、Matplotlib 与 Pillow 后，可依次运行主结果和实验程序。完整统一入口为：

```
python -X utf8 reproduce.py --project-root .
```

该入口从原始附件读取数据，依次运行测试、四题结果、54 组实验、工作簿生成、图表输出和格式检查。跨机器使用时，应将绘图辅助工具路径配置为本地实际位置；如只需核对已有输出，可运行各审计程序而不重复年度优化。